



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Mémoire de Licence

Filière : Informatique

Spécialité : Informatique Générale

Thème

Apprentissage automatique et profond pour l'optimisation
multi-objectifs de la sélection des services IoT dans le cloud

Sous la direction de :
Mr. BOUDOUR Rachid
Mr. YAKLOUF Youcef

Présenté par :
FEKIR Nabila Chehla
BENMESSAI Sarah

Membres du jury :
Mme. BOUIBEDE Karima (Présidente)
Mme. BENSEBAA Amina (Membre)

Remerciement

Au nom de notre binôme, nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui nous ont soutenus et accompagnés tout au long de notre parcours.

Tout d'abord, nous adressons nos remerciements à Allah, pour nous avoir guidés et accordés la force nécessaire pour mener à bien ce travail de recherche.

Nous exprimons également notre reconnaissance envers notre encadrant Mr.Boudour R., pour sa patience, son expertise et ses conseils avisés qui ont grandement contribué à l'élaboration de ce mémoire. Sa disponibilité et ses encouragements nous ont été d'une aide précieuse.

À nos chers parents, nos frères et sœurs et toute nos familles, nous sommes infiniment reconnaissants pour leur soutien indéfectible, leur amour inconditionnel et leurs encouragements constants. Leur confiance en nous a été une source de motivation inestimable.

Nous tenons également à remercier nos amies pour leur soutien moral, leurs encouragements et leur présence tout au long de cette aventure. Leur soutien nous a permis de surmonter les moments difficiles avec plus de sérénité.

Enfin, nous adressons nos sincères remerciements à Monsieur Zebouchi,A. pour son aide précieuse et ses précieux conseils qui ont enrichi notre travail.

À chacun d'entre vous, nous vous témoignons notre profonde reconnaissance et notre gratitude sincère. Vos contributions ont été essentielles à la réalisation de ce mémoire.

Résumé

L'intégration généralisée de l'Internet des objets (IoT) parvient à transformer divers secteurs, notamment celui de la santé. Avec l'augmentation des objets connectés et des données qu'ils génèrent, le problème de sélection des services IoT dans le cloud est apparu. Chaque service comporte un ensemble de qualités de service (QoS) définies, qui le décrivent selon plusieurs critères, tout en prenant en considération l'éventuelle absence du service demandé, c'est là qu'intervient la proposition d'un ensemble de services dans un environnement multi-cloud appelé composition de service, ayant une fonction aussi proche que possible de celle du service demandé. Le problème de sélection multi-objectifs, étant un problème combinatoire, le rend complexe et impossible à résoudre avec des approches exactes traditionnelles. La majorité des solutions proposées dans la littérature tournent autour de l'utilisation d'algorithmes métaheuristiques multi-objectifs pour trouver les solutions les plus optimales. Notre contribution réside dans l'intégration des techniques d'apprentissage automatique et profond par renforcement, un domaine de recherche prometteur, pour résoudre ce problème. Nous avons réalisé cela à travers l'implémentation de deux algorithmes d'apprentissage multi-objectifs par renforcement, MPMOQL et PCN, avec lesquels nous avons obtenu des résultats satisfaisants. Nous envisageons de gérer les limites constatées au cours de cette recherche afin d'améliorer la solution proposée.

Mots Clés : Apprentissage automatique, Apprentissage profond, Apprentissage par renforcement, Multi-objectif, services IoT, Cloud computing, Composition, SOA, Selection de services.

Abstract

The widespread integration of the Internet of Things (IoT) is transforming various sectors, notably healthcare. With the increase in connected devices and the data they generate, the issue of IoT service selection in the cloud has emerged. Each service has a set of defined Quality of Service (QoS) attributes that describe it according to several criteria, while also considering the potential absence of the requested service. This is where the proposal for a set of services in a multi-cloud environment, called service composition is introduced, aiming to provide functionality as close as possible to that of the requested service. The multi-objective selection problem, being a combinatorial issue, is complex and impossible to solve with traditional exact approaches. Most of the solutions proposed in the literature revolve around the use of multi-objective metaheuristic algorithms to find the most optimal solutions. Our contribution lies in integrating machine and deep reinforcement learning techniques, a promising research area, to address this problem. We achieved this through the implementation of two multi-objective reinforcement learning algorithms, MPMOQL and PCN, with which we obtained satisfactory results. We plan to address the limitations identified during this research to further improve the proposed solution.

Keywords: Machine learning, Deep learning, Reinforcement learning, Multi-objective, IoT services, Cloud computing, Composition, SOA, Service selection.

ملخص

ساهم الإدماج الشامل لخدمات إنترنت الأشياء المتصلة بدعم من تكنولوجيا الحوسبة السحابية في إحداث قفزة نوعية في مختلف القطاعات بما في ذلك قطاع الصحة، لكن و مع زيادة الأجهزة المتصلة والبيانات التي تولدها، واجهت المستخدمين صعوبة إيجاد وإختيار أفضل الخدمات المتاحة. تحتوي كل خدمة على مجموعة محددة من صفات الجودة يتم على اساسها تقييم الخدمة (السعر، السرعة، الخ). في حال عدم توفر خدمة من الخدمات المطلوبة، يأتي إقتراح دمج مجموعة من الخدمات في بيئة متعددة السحب أو ما يسمى بالخدمة المركبة، هذه التركيبة تؤدي الى حد ما نفس وظيفة الخدمة المطلوبة. هنا تكمن مشكلة الاختيار متعدد الأهداف لضمان أكبر قدر ممكن من جودة الخدمة المختارة، كونها مشكلة تركيبية، ما يجعلها معقدة وغير قابلة للحل بالطرق التقليدية المباشرة. تكمن مساهمتنا في دمج تقنيات التعلم الآلي والعميق المعزز متعدد الأهداف في هذا المجال، ما أتاح لنا الحصول على نتائج مرضية الى حد بعيد من حيث تحسين إختيار أفضل الخدمات. وفي الختام، نظرا لتسجيل بعض النقائص، يفتح نظامنا آفاقا جديدة للتعامل مع مشكلة الاختيار متعدد الأهداف في عالم الأجهزة المتصلة بالإنترنت، من خلال تحسين بعض جوانب التعلم الآلي المعزز للتكيف بشكل افضل في هذه البيئة.

الكلمات المفتاحية : التعلم الآلي، التعلم العميق، التعلم المعزز متعدد الأهداف، خدمات الإنترنت للأشياء، الحوسبة السحابية، تركيب الخدمات، اختيار الخدمات..

Table des matières

Table des figures	i
Liste des tableaux	ii
Liste des Algorithmes	iii
Introduction Générale	1
1 Généralités	3
1.1 Introduction	3
1.2 Processus d'optimisation	3
1.2.1 Types d'optimisation	3
1.2.2 Description formelle de l'optimisation multi-objectifs	4
1.2.3 Buts de l'optimisation multi-Objectifs	5
1.3 Apprentissage automatique (Machine Learning)	5
1.3.1 Définition	5
1.3.2 Apprentissage profond (Deep Learning)	6
1.3.3 Apprentissage par renforcement	6
1.3.4 Apprentissage profond par renforcement	8
1.4 Apprentissage par renforcement multi-objectifs	9
1.4.1 Processus décisionnel multi-objectifs de Markov (MOMDP)	9
1.4.2 Fonction de scalarisation (fonction d'utilité)	10
1.4.3 Apprentissage par renforcement à politiques multiples	11
1.5 Conclusion	11
2 État de l'art	12
2.1 Introduction	12
2.2 Internet des objets (IoT)	12
2.2.1 Architecture des IoTs	12
2.2.2 Qualités de services (QoS)	13

2.2.3	Internet des objets médicaux (IoMT)	14
2.3	Cloud Computing	14
2.3.1	Modèles de services	14
2.3.2	Modèles de déploiement	15
2.3.3	Architecture orientée services (SOA)	16
2.4	Définition du problème	16
2.5	Approches de l'optimisation multi-objective	16
2.5.1	Optimisation via les métaheuristiques	17
2.5.2	Optimisation via l'apprentissage par renforcement	19
2.6	Comparaison des approches d'optimisation	21
2.7	Conclusion	22
3	Conception	23
3.1	Introduction	23
3.2	Architecture globale du système	23
3.3	Formulation du problème	24
3.3.1	Codage de la solution	24
3.3.2	Prétraitement des données	25
3.3.3	Conception de l'environnement	25
3.4	Algorithme multi-objets à politiques multiples par Q-Learning (MPMOQL)	28
3.4.1	Algorithme Q-Learning	28
3.4.2	Algorithme MPMOQL	28
3.5	Algorithme Réseaux conditionnés par Pareto (PCN)	29
3.5.1	Définition	29
3.5.2	Dataset des Expériences de l'Agent	29
3.5.3	Entraînement de l'algorithme	30
3.6	Conclusion	30
4	Implémentation de la solution	31
4.1	Introduction	31
4.2	Environnement de travail	31
4.2.1	Environnement matériel :	31
4.2.2	Environnement logiciel	31
4.3	Dataset	33
4.4	Implémentation	33
4.4.1	Hyperparamètres	34
4.5	Indicateurs de performances	35

4.6	Tests et analyse	35
4.7	Discussion	37
4.8	Démonstration	37
4.9	Conclusion	39
Bibliographie		42
A Implementation de l’algorithme NSGA-III		A
A.0.1	Paramètres	A
A.0.2	Métriques	A
A.0.3	Graphe de nuage de points par paire	B
A.0.4	Comparaison	D
B Tests supplémentaires		E
B.0.1	Métriques	E
B.0.2	Graphe de nuage de points par paire	G
C Algorithme Envelope		I
C.1	Hyperparamètres	I
C.2	Tests et analyse	I
C.3	Discussion	J

Table des figures

1.1	Illustration des points dominants et dominés du front de Pareto pour problème de minimisation à deux objectifs. [1]	4
1.2	Interaction entre l'agent et l'environnement [2]	7
1.3	Réseau de neurones avec fonction de valeurs en sortie [3]	9
1.4	Comparaison entre MDP et MOMDP [4]	10
1.5	Illustration du CCS et representation des préférences	11
2.1	Couches de l'architecture de l'IoT [5]	13
2.2	Les modèles de services Cloud [6]	15
2.3	Procédure de l'algorithme NSGA-II [7]	18
3.1	Architecture de composition et sélection des services dans le multi-cloud [8]	24
3.2	Exemple d'une composition de quatre services	24
3.3	Exemple illustrant l'interaction de l'agent avec son environnement.	27
3.4	Table de Q-valeurs dans l'algorithme Q-Learning [9]	28
4.1	Interface conçue pour choisir les services de la composition	38
4.2	Interface conçue pour afficher la composition sélectionnée	39
A.1	Graphe de nuage de points par paire	C
B.1	Graphique représentant l'hypervolume de dix compositions différentes.	F
B.2	Graphique représentant l'IGD de dix compositions différentes	F
B.3	Graphique représentant la cardinalité de dix compositions différentes	G
B.4	Graphe de nuage de points par paire	H
C.1	Hypervolume et Cardinalité de l'algorithme Envelope	J

Liste des tableaux

2.1	Avantages et limitations des différentes approches de sélection de services IoT dans le cloud.	21
4.1	Exemple de données de quelques services dans le multi-cloud à améliorer. . .	33
4.2	Hyperparamètres attribués pour les algorithmes MPMOQL et PCN	35
4.3	Métriques obtenues lors de l'exécution des algorithmes MPMOQL et PCN.	36
A.1	Paramètres attribués au NSGA-III	A
C.1	Hyperparamètres attribués pour l'algorithme Envelope	I

Liste des Algorithmes

3.1	Algorithme Q-Learning Multi-Objectifs	29
-----	---	--------------------

Introduction Générale

L'Internet des objets (IoT) représente un système composé de dispositifs et de machines capables de générer et de transférer des données à travers le réseau, ces données évoluent constamment, en parallèle avec les avancées des systèmes IoT, c'est pourquoi les services Cloud jouent un rôle indispensable en facilitant la circulation des données après que l'architecture orientée service (SOA) nous a permis de passer d'une conception matérielle physique d'un objet à une vision orientée service.

De nos jours, avec l'expansion croissante de l'utilisation des services IoT dans notre vie quotidienne, un seul cloud avec ses ressources limitées est devenu insuffisant pour répondre à toutes les demandes des utilisateurs et garantir une meilleure qualité de service (QoS). Ainsi, la coopération entre plusieurs clouds dans un environnement multi-cloud apparaît comme une solution plus efficace, permettant aux clouds de communiquer entre eux.

Les clients n'ont qu'à envoyer leur requête pour trouver le service approprié, pour répondre à un éventuel manque de service adapté à leurs besoins, une des solutions consiste à combiner des services sélectionnés parmi différents clouds, créant ainsi une composition de services. permettant d'obtenir une fonctionnalité similaire à ce qui est recherché.

La sélection de la composition de services adaptée aux besoins des clients constitue le principal défi abordé dans ce mémoire. Il est essentiel de souligner que les besoins des clients peuvent parfois être contradictoires, tels que la nécessité de réduire à la fois le temps de réponse et le coût, cela demande de trouver un équilibre entre différents objectifs à travers l'optimisation multi-objectifs, en tenant compte principalement des préférences variées des utilisateurs en termes de qualité de service (QoS). Par exemple, certains utilisateurs peuvent privilégier un temps de réponse réduit même si cela entraîne des coûts plus élevés.

Pour résoudre ce problème plusieurs approches ont été proposées, la plus récente étant l'utilisation de l'apprentissage automatique et profond. L'objectif principal de ce travail est donc de proposer une solution au problème d'optimisation de la sélection des services IoT dans un environnement multi-Cloud en utilisant des algorithmes multi-objectifs d'apprentissage par renforcement (MORL), en particulier dans le domaine de la santé.

Dans le premier chapitre, nous avons commencé par définir l'optimisation dans le contexte multi-objectifs, puis nous avons présenté des généralités et quelques notions de base sur le processus d'apprentissage automatique, profond, et également l'apprentissage par renforcement, avec ses deux variantes mono et multi-objectifs.

Dans le deuxième chapitre, nous aborderons les systèmes IoT et leurs architectures, ainsi que le cloud computing, ses modèles de services et de déploiement. Nous passerons également en revue quelques approches proposées pour résoudre la problématique énoncée, en expliquant le principe de fonctionnement de chaque technique et en résumant les avantages et les inconvénients de ces méthodes dans un tableau Comparatif.

Dans le troisième chapitre, nous introduirons la conception détaillée de notre système, qui tourne autour de l'environnement conçu pour appliquer les algorithmes d'apprentissage multi-objectifs par renforcement (MORL). Nous expliquerons notre espace de recherche et notre fonction de récompense, ainsi que les algorithmes que nous avons implémentés : l'algorithme multi-objectifs à politiques multiples par Q-Learning (MPMOQL) et les réseaux conditionnés par Parto (PCN).

Dans le quatrième chapitre, nous présenterons la mise en œuvre de notre approche, nous commencerons par la présentation des outils et de l'environnement de développement, ainsi que notre dataset et nos méthodes d'évaluation. Ensuite, nous présenterons l'interface conçue pour illustrer les résultats obtenus.

Enfin, à la clôture de ce mémoire, une conclusion générale sera présentée incluant quelques perspectives découlant de ce travail.

Chapitre 1

Généralités

1.1 Introduction

Dans un monde en constante évolution, l'apprentissage automatique est devenu un élément indispensable, exerçant une influence considérable sur nos expériences dans la vie moderne, notamment en ce qui concerne les problèmes d'optimisation. Dans ce chapitre, nous aborderons des concepts clés des systèmes d'optimisation basés sur les techniques de l'apprentissage automatique. Nous entamerons ce chapitre par le processus d'optimisation multi-objectifs, puis aborderons l'apprentissage automatique, profond et par renforcement, pour conclure enfin avec l'apprentissage par renforcement multi-objectifs.

1.2 Processus d'optimisation

Un problème d'optimisation consiste à maximiser ou à minimiser une ou plusieurs fonctions d'objectif en choisissant systématiquement des valeurs d'entrée appelées variables de décision au sein d'un ensemble. ce qui revient à rechercher la solution optimale parmi un ensemble de solutions potentielles [10].

1.2.1 Types d'optimisation

On peut catégoriser l'optimisation selon le nombre d'objectifs qu'elle vise à atteindre comme suit :

1.2.1.1 Optimisation mono-objectif

Consiste à rechercher la solution optimale qui fournit la valeur la plus élevée ou la plus faible de la fonction objectif [11]. Cependant, dans la réalité, les problèmes posés

sont souvent plus complexes et impliquent plusieurs objectifs à prendre en compte. C'est précisément pour cette raison que l'optimisation multi-objectifs existe.

1.2.1.2 Optimisation multi-objectifs

Également connue sous le nom d'optimisation de Pareto, vise à maximiser ou minimiser plusieurs objectifs simultanément dans le processus de prise de décision, elle cherche à trouver un compromis optimal entre ces objectifs, même s'ils peuvent être en conflit, par exemple minimiser le temps de réponse tout en maximisant la fiabilité ou maximiser les performances tout en minimisant les coûts [12].

Contrairement à l'optimisation à objectif unique, où la meilleure solution est facilement identifiable, dans l'optimisation multi-objectifs, la qualité d'une solution est déterminée par sa domination par rapport aux autres solutions. Un ensemble de solutions non dominées est représenté par le front de Pareto *illustré dans la figure ci-dessous*, où aucune solution ne peut être améliorée pour tous les objectifs sans sacrifier au moins un autre.

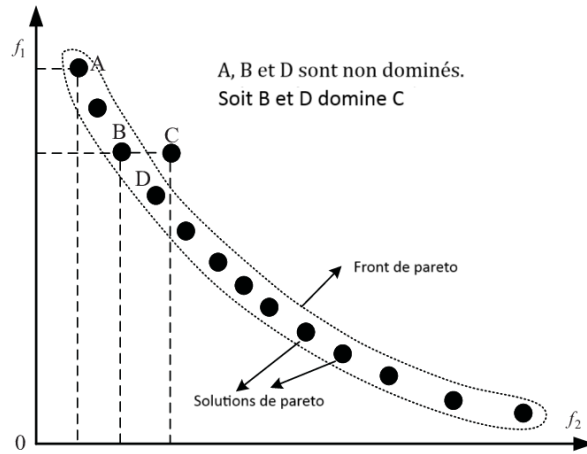


Figure 1.1: Illustration des points dominants et dominés du front de Pareto pour problème de minimisation à deux objectifs. [1]

1.2.2 Description formelle de l'optimisation multi-objectifs

Le problème d'optimisation multi-objectifs peut être mathématiquement représenté de la manière suivante [13] :

$$\min_{\mathbf{x}} \mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x})) \quad \text{s.t.} \quad \mathbf{x} \in \mathbf{X} \quad (1.1)$$

Où $\mathbf{f}(\mathbf{x})$ est composée de M différentes fonctions d'objectif et $\mathbf{X} \subseteq \mathbb{R}_D$ est l'espace de décision. Si l'un des objectifs doit être maximisé, sa fonction d'objectif est multipliée

par-1, afin de pouvoir minimiser l'ensemble.

On dit que la solution A domine la solution B, si cette équation est vérifiée [13] :

$$A \succ B \Leftrightarrow \forall i \in 1, 2, \dots, M \quad f_i(A) \geq f_i(B) \quad \text{et} \quad \exists j \in 1, 2, \dots, M \quad f_j(A) > f_j(B) \quad (1.2)$$

1.2.3 Buts de l'optimisation multi-Objectifs

Fondamentalement, l'optimisation multi-objectifs vise à atteindre deux objectifs principaux :

- Trouver un ensemble de solutions aussi proche que possible du front de Pareto optimal.
- Trouver un ensemble de solutions aussi diversifié que possible, ce qui aide à examiner un plus grand nombre de possibilités et à donner plus d'options aux décideurs.

1.3 Apprentissage automatique (Machine Learning)

1.3.1 Définition

L'apprentissage automatique est un domaine de recherche qui constitue une branche de l'intelligence artificielle. Il se focalise sur l'utilisation de méthodes statistiques pour permettre aux machines d'apprendre à partir de données, ce qui leur permet de s'améliorer au fil du temps sans être explicitement programmées pour chaque tâche [14].

Types d'apprentissage automatique

Il existe diverses catégories d'apprentissage automatique, incluant les types suivants :

- **Apprentissage supervisé** : Consiste à utiliser des données étiquetées pour entraîner un algorithme, visant à prédire précisément les étiquettes pour de nouvelles données.
- **Apprentissage non supervisé** : Se base sur des données non étiquetées, permettant à l'algorithme de découvrir des motifs et des relations, souvent utilisé pour regrouper des données similaires en identifiant les caractéristiques importantes.
- **Apprentissage semi-supervisé** : Utilise à la fois des données étiquetées et non étiquetées.
- **Apprentissage par renforcement** : Implique l'apprentissage d'une politique par un algorithme qui permet de prendre des décisions séquentielles dans son environnement à travers l'accumulation des récompenses.

1.3.2 Apprentissage profond (Deep Learning)

L'apprentissage profond est une composante de l'apprentissage automatique qui utilise des algorithmes connus sous le nom de réseaux de neurones. En général, on fait appel à l'apprentissage profond lorsque les caractéristiques d'un ensemble de données doivent être extraites automatiquement. Il est souvent utilisé pour les problèmes avec des données non structurées ou pour des problèmes complexes nécessitant un très grand ensemble de données. L'apprentissage profond nécessite souvent des ressources coûteuses telles que des unités de traitement graphique (GPU) [15].

Réseaux de neurones

Les réseaux de neurones sont des algorithmes inspirés du cerveau humain, composés de neurones organisés en couches, incluant une couche d'entrée, une couche de sortie et des couches intermédiaires appelées couches cachées, permettant de capturer des caractéristiques complexes de données.

Chaque neurone a un poids qui influence son entrée et un biais pour introduire un décalage dans la sortie. En ajustant itérativement ces poids et biais sur un ensemble de données, le réseau est entraîné pour effectuer des prédictions sur de nouvelles données.

Les fonctions d'activation telles que la sigmoïde, ReLU et tanh sont utilisées pour permettre au réseau de capturer des relations non linéaires entre les données, améliorant ainsi sa capacité de modélisation.

1.3.3 Apprentissage par renforcement

L'apprentissage par renforcement constitue une branche essentielle de l'apprentissage automatique, permettant à un modèle d'effectuer des prises de décision séquentielles et autonomes. Au cœur de ce processus, un agent interagit avec son environnement, apprenant progressivement à travers l'accumulation de récompenses à chaque action prise à un instant et dans un état donné, l'idée centrale est de découvrir une stratégie ou politique qui guide l'agent à choisir des actions menant aux résultats les plus favorables dans son environnement [14].

1.3.3.1 Processus de décision markov (MDP)

Le processus de décision markov fournit une notation formelle pour décrire comment un agent prend des décisions en interagissant avec son environnement. Voici ses éléments clés :

- **L'agent** : L'acteur qui agit et prend des décisions.

- **L'état** : Les situations différentes où l'agent peut se trouver.
- **L'action** : La décision prise par l'agent à un état donné.
- **Les récompenses** : Des valeurs numériques qui représentent le niveau de bénéfice associé à l'action sélectionnée par l'agent dans un état donné.
- **L'environnement** : Le cadre dans lequel l'agent opère et prend des décisions.

À chaque étape temporelle t , l'agent reçoit un état s_t , choisit une action a_t en suivant une politique $\pi(a_t, s_t)$ qui représente son comportement, puis obtient une récompense r_t en retour. Par la suite, il transitionne vers l'état suivant s_{t+1} selon une probabilité de transition $p(s_{t+1} | s_t, a_t)$. Ce cycle se répète jusqu'à ce que l'agent atteigne un état final. Le gain est une récompense accumulée pondérée par un facteur de réduction γ afin d'équilibrer les récompenses à court et à long terme et garantir la convergence des algorithmes. L'objectif de l'agent est donc de maximiser son gain illustré comme suit [16].

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \quad \text{où } \gamma \in (0, 1] \quad (1.3)$$

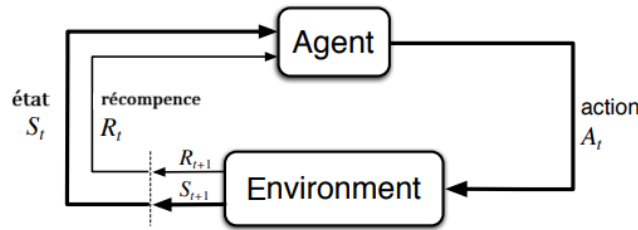


Figure 1.2: Interaction entre l'agent et l'environnement [2]

1.3.3.2 Politique d'apprentissage (policy)

Une politique est définie comme une fonction $\pi: S \times A \rightarrow [0, 1]$, où pour tout état donné, une action est sélectionnée selon une certaine distribution de probabilité.

L'objectif de l'agent est de trouver une politique optimale qui représente une stratégie visant à maximiser l'accumulation de récompenses.

1.3.3.3 Fonctions de valeur

Les fonctions de valeur [16] prédisent les récompenses futures accumulées et actualisées. On distingue deux types de fonctions de valeur :

- **Valeur de l'état (state value)** : Renseigne sur la récompense attendue selon une politique donnée dans un état donné, permettant ainsi d'évaluer l'utilité de chaque

état pour l'agent.

$$V_\pi(s) = \mathbb{E}_\pi [R_t | s_t = s] \quad (1.4)$$

La fonction de valeur maximale d'état, obtenue sous n'importe quelle politique pour un état s , est appelée fonction d'état optimale, obtenue à travers l'équation de Bellman suivante :

$$V_*(s) = \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V_*(s')] \quad (1.5)$$

- **Valeur de l'action-état (state-action value) :** Indique la récompense attendue en suivant une politique dans un état donné tout en choisissant une action spécifique. Cette fonction nous aide à évaluer l'efficacité de chaque paire état-action pour l'agent.

$$Q_\pi(s, a) = \mathbb{E}_\pi [R_t | s_t = s, a_t = a] \quad (1.6)$$

De même, La fonction de valeur maximale d'un état-action paire, obtenue sous n'importe quelle politique pour un état s et une action a , est appelée fonction d'état-action optimale, obtenue à travers l'équation de Bellman suivante :

$$Q_*(s) = \max_a \sum_{s',r} p(s', r | s, a) [r + \max_{a'} \gamma Q_*(s', a')] \quad (1.7)$$

1.3.3.4 Exploration et exploitation

La performance de l'agent dépend largement de sa capacité à explorer son environnement pour recueillir des informations, qu'il pourra ensuite utiliser pour sélectionner les actions maximisant les récompenses [17]. Le principal défi consiste à équilibrer une exploration rapide avec une exploitation efficace de l'environnement [18].

Epsilon-greedy : Une stratégie simple pour équilibrer l'exploration et l'exploitation. Avec une probabilité $\epsilon \in [0, 1]$, l'agent choisit d'explorer son environnement en sélectionnant une action de manière aléatoire afin d'acquérir une nouvelle expérience. Avec une probabilité de $1 - \epsilon$, l'agent choisit d'exploiter ses expériences en sélectionnant l'action qui lui a apporté la meilleure récompense. Au fur et à mesure que l'agent interagit avec son environnement, ϵ diminue afin de privilégier l'exploitation.

1.3.4 Apprentissage profond par renforcement

L'apprentissage profond par renforcement fusionne les principes de l'apprentissage par renforcement avec les techniques de l'apprentissage profond. L'apprentissage par renforcement se révèle efficace pour la prise de décisions séquentielles, tandis que l'apprentissage

profond excelle dans la modélisation de représentations complexes et l'extraction de caractéristiques. Cette approche permet à l'agent d'interagir et d'apprendre à partir d'environnements plus complexes, tout en utilisant un réseau de neurones qui prend en entrée son état actuel s et fournit en sortie une approximation des fonctions de valeur ou de politique. C'est ainsi que l'agent choisit l'action qui lui apportera le plus de récompenses [16].

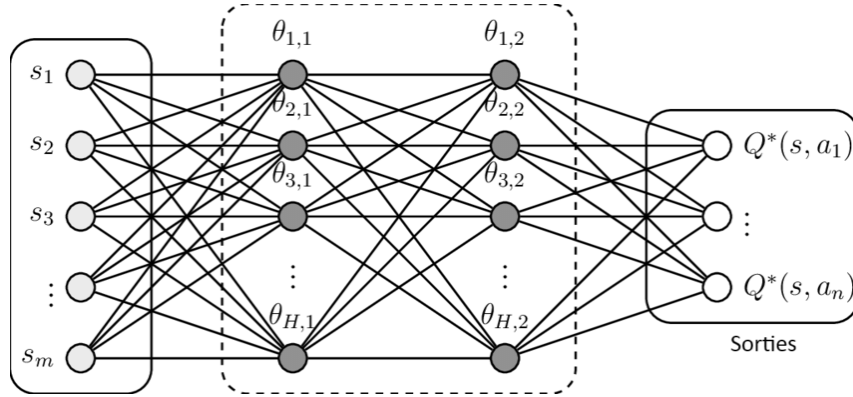


Figure 1.3: Réseau de neurones avec fonction de valeurs en sortie [3]

1.4 Apprentissage par renforcement multi-objectifs

Il s'agit d'une variante de l'apprentissage par renforcement qui prend en compte plusieurs objectifs simultanément, en attribuant des récompenses distinctes à l'agent pour chaque objectif. Le but n'est plus simplement de maximiser la récompense cumulative attendue, mais de trouver l'ensemble du front de Pareto, c'est-à-dire les solutions dominantes.

1.4.1 Processus décisionnel multi-objectifs de Markov (MOMDP)

Un MDP comme précédemment expliqué, est un cadre séquentiel de prise de décision pour un problème à objectif unique. La différence entre un MDP à objectif unique et un MOMDP réside dans le fait que la récompense attendue n'est pas un scalaire, mais plutôt un vecteur où chaque élément représente la récompense obtenue pour un objectif spécifique. Étant donné que les récompenses sont des vecteurs, les valeurs de l'état sont également des vecteurs.

$$\vec{R}(s_i, a_i) = (R_1(s_i, a_i), R_2(s_i, a_i), \dots, R_d(s_i, a_i)), \quad d \text{ est le nombre d'objectifs} \quad (1.8)$$

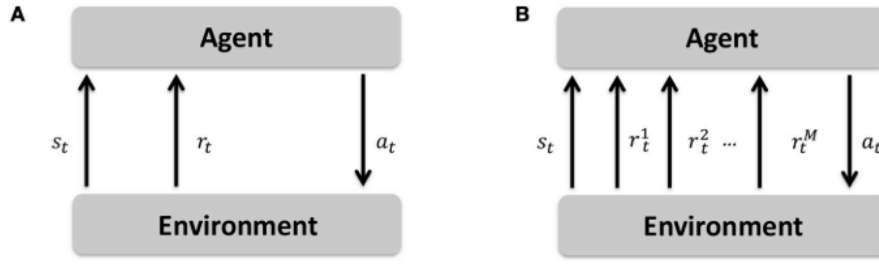


Figure 1.4: Comparaison entre MDP et MOMDP [4]

(A) Processus décisionnel de Markov (MDP) et (B) Processus décisionnel de Markov multi-objectifs (MOMDP).

Éléments du MOMDP

Un MOMDP peut être représenté par le tuple $(S, A, P, r, \Omega, f_\Omega)$ [19].

- S L'espace des états.
- A L'espace d'actions.
- $P(s'|s, a)$ La distribution de transition.
- $r(s, a)$ Le vecteur de récompense.
- Ω L'espace des préférences.
- $f_\omega(r)$ Produit une utilité scalaire en utilisant les préférences $\omega \in \Omega$.

1.4.2 Fonction de scalarisation (fonction d'utilité)

L'optimisation multi-objectifs par renforcement nous fournit un ensemble de politiques optimales plutôt qu'une seule. Une fonction d'utilité joue un rôle essentiel en sélectionnant la politique la plus adaptée, en tenant compte des préférences de l'utilisateur.

$u: \mathbb{R}^d \rightarrow \mathbb{R}$, mappe la valeur multi-objectifs d'une politique à une valeur scalaire.

$$\mathbf{V}_u^\pi = u(\mathbf{V}^\pi) \quad (1.9)$$

Fonction d'utilité linéaire : Dans le cadre de ce projet, nous nous intéressons à une fonction d'utilité linéaire qui calcule le produit scalaire d'un vecteur de poids ω et d'un vecteur de valeurs $\mathbf{V}^\pi$.

$$u(\mathbf{V}^\pi) = \omega^\top \mathbf{V}^\pi \quad (1.10)$$

Chaque composante du vecteur de poids ω indique la contribution d'une unité de valeur pour l'objectif correspondant. Les éléments de ce dernier sont des réels positifs et leur somme est égale à 1.

1.4.3 Apprentissage par renforcement à politiques multiples

Dans les MOMDPs l'agent se comporte selon une politique $\pi \in \Pi$, où Π est l'ensemble de toutes les politiques possibles. Pour toutes les préférences possibles on définit le CCS (convex coverage set) qui est un sous-ensemble de la frontière de Pareto définie avec l'équation suivante.

$$\mathcal{F}^* := \{\hat{r} \mid \nexists \hat{r}' \geq \hat{r}\} \quad (1.11)$$

Le CCS représente toutes les solutions sur sa frontière convexe externe, et qui contient toutes les récompenses fournissant l'utilité cumulée maximale [19].

$$CCS := \{\hat{r} \in \mathcal{F}^* \mid \exists \omega \in \Omega \text{ s.t. } \omega^\top \hat{r} \geq \omega^\top \hat{r}', \forall \hat{r}' \in \mathcal{F}^*\} \quad (1.12)$$

Le but alors est d'entraîner l'agent sur plusieurs politiques afin de couvrir la totalité de l'ensemble du CCS, puis s'adapter à une politique optimale pour toute préférence donnée.

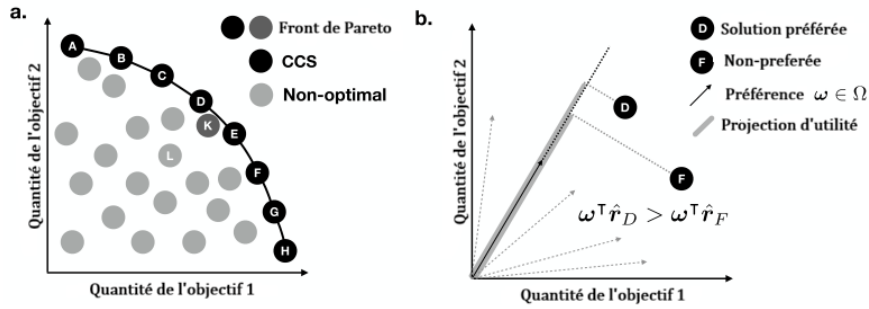


Figure 1.5: Illustration du CCS et représentation des préférences

(a) La frontière de Pareto pourrait ne pas être convexe. Dans cet exemple, le CCS comprend les points A à H mais exclut K, alors que tous les points de A à K forment la frontière de Pareto. (b) étant donné un vecteur de préférence ω , la solution optimale sera un point sur le CCS qui maximise la projection sur ω [19].

1.5 Conclusion

Dans ce chapitre, nous avons introduit les concepts clés liés à notre projet, notamment l'optimisation multi-objective, l'apprentissage automatique et profond. Nous avons également abordé les bases de l'apprentissage automatique. Ensuite, nous avons examiné de manière approfondie l'apprentissage par renforcement multi-objectifs, un domaine de recherche récent et très prometteur. Dans le prochain chapitre, nous analyserons en profondeur les différentes approches employées pour résoudre le problème de la sélection des services IoT dans le cloud. en mettant l'accent sur les méthodes basées sur les métaheuristiques multi-objectifs ainsi que sur l'apprentissage par renforcement à objectif unique.

Chapitre 2

État de l’art

2.1 Introduction

Aujourd’hui, l’Internet des Objets (IoT) est largement intégré dans le domaine des soins de santé, avec une multitude d’appareils connectés qui génèrent souvent des données hébergées chez différents fournisseurs de cloud, nécessitant ainsi un traitement sur multicloud. Pour relever le défi de sélectionner les services IoT les plus adaptés, il est indispensable de trouver un équilibre entre différents objectifs afin de répondre aux besoins divers des utilisateurs. Dans ce chapitre, nous introduirons les systèmes IoT et les notions principales du cloud, puis nous aborderons également le problème en détail et présenterons l’état de l’art du problème.

2.2 Internet des objets (IoT)

L’IoT, ou Internet des Objets, désigne la capacité de connecter tout ce qui nous entoure en termes de matériel, comme les machines, les appareils, les téléphones mobiles et les voitures, etc., à Internet avec un comportement intelligent, en tenant compte à la fois de l’autonomie et de la sécurité des informations. [20]

2.2.1 Architecture des IoTs

Diverses architectures IoT ont été proposées dans la littérature, variant d’une structure à trois couches à une structure à sept couches. À titre d’exemple, nous examinerons une architecture à quatre couches, comme illustré dans la figure ci-dessous : la détection, le réseau, le cloud computing et les applications. Ces couches sont conçues pour relever les défis des systèmes IoT.

- **Couche de détection** : Se concentre sur la collecte de données à partir de différents capteurs et appareils physiques [21].
- **Couche de réseau** : Gère la communication et la connectivité entre les appareils et le cloud [21].
- **Couche du cloud computing** : Responsable du stockage, du traitement et de l'analyse des données [21].
- **Couche des applications** : Permet le développement et le déploiement des applications IoT [21].

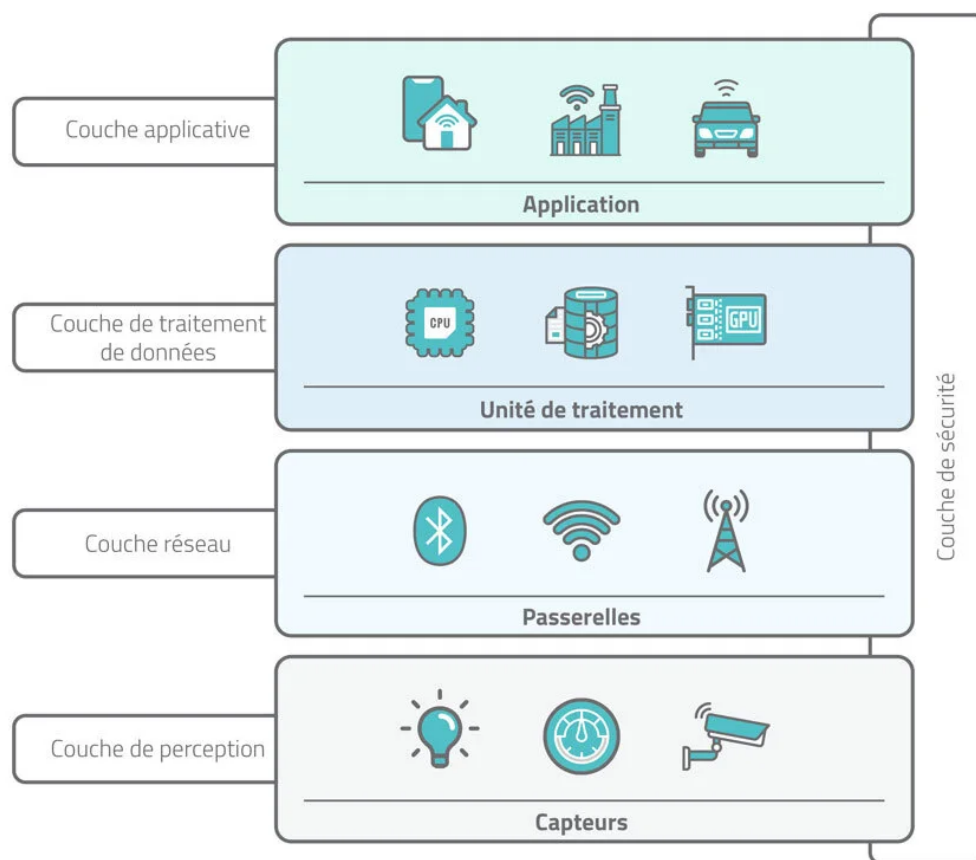


Figure 2.1: Couches de l'architecture de l'IoT [5]

2.2.2 Qualités de services (QoS)

La Qualité de Service (QoS) désigne le niveau mesurable de performance et de fiabilité qu'un fournisseur de services garantit à ses clients, les principales QoS sont :

- **Consommation d'énergie** : Cette qualité de service est importante en raison de la dépendance des objets à l'électricité. Les fournisseurs des services cherchent donc à

réaliser ces tâches en utilisant le moins d'énergie possible tout en étant efficaces [8].

- **Coût** : Avec l'augmentation du nombre de fournisseurs du cloud computing et de services, le coût de certains services est devenu un élément clé des qualités de service (QoS). Ainsi, la minimisation du coût est devenu un objectif considérablement important [8].
- **Temps de réponse** : C'est le temps nécessaire pour envoyer une requête et recevoir la réponse du service [8].
- **Nombre de cloud dans une composition** : Le nombre de clouds utilisés dans la composition de services, ainsi que l'augmentation de la communication inter-cloud, entraîne une hausse de la consommation d'énergie et des coûts de communication [8].

2.2.3 Internet des objets médicaux (IoMT)

Les technologies de l'IoT ouvrent un large éventail de possibilités pour améliorer les soins de la santé, en prévoyant et diagnostiquant les problèmes de santé, traitant et surveillant les patients à l'intérieur et à l'extérieur de l'hôpital. Il est de plus en plus important de comprendre comment ces technologies IoT peuvent aider les systèmes de santé à offrir des soins sûrs et efficaces [22]. Par exemple les systèmes de Surveillance Continue de la Glycémie (CGM) surveillent en continu les niveaux de glucose dans le sang, fournissant ainsi aux patients et aux médecins des données en temps réel sur la glycémie [23].

2.3 Cloud Computing

Le Cloud Computing est un modèle qui permet aux utilisateurs d'accéder via Internet à des ressources informatiques à la demande, telles que des applications, des serveurs (physiques et virtuels) et du stockage de données [24]. Ces ressources sont hébergées dans un centre de données distant géré par un fournisseur des services cloud, sans que l'utilisateur ait besoin de manipuler ou gérer une infrastructure complexe. En somme, les clients bénéficient de ces services sous forme d'abonnements flexibles, adaptés à leur utilisation [25, 26].

2.3.1 Modèles de services

Il existe principalement quatre modèles de services, *illustrés par la figure ci-dessous* :

- **IaaS (Infrastructure as a Service)** : Un modèle IaaS permet aux utilisateurs d'accéder facilement à des systèmes complets. En d'autres termes, ce dernier fournit

les éléments matériels de base de l'infrastructure, auxquels les clients peuvent accéder directement [27, 28].

- **PaaS (Platform as a Service)** : Dans ce modèle, les développeurs peuvent créer et tester leurs logiciels sans avoir besoin d'installer des outils spéciaux sur leur propre ordinateur, et ce sont les fournisseurs qui se chargent de mettre à jour et de maintenir en bon état l'infrastructure cloud [28, 29].
- **SaaS (Software as a Service)** : Ce modèle propose des logiciels en ligne prêts à l'emploi. Le fournisseur a le contrôle complet sur le logiciel [28, 29].
- **XaaS (Anything as a service)** : Ce modèle désigne la variété croissante de services accessibles sur Internet via le Cloud Computing. Cette terminologie englobe plusieurs modèles en cours de développement, regroupant tous ces services sous la dénomination générale de "n'importe quoi en tant que service" [28].

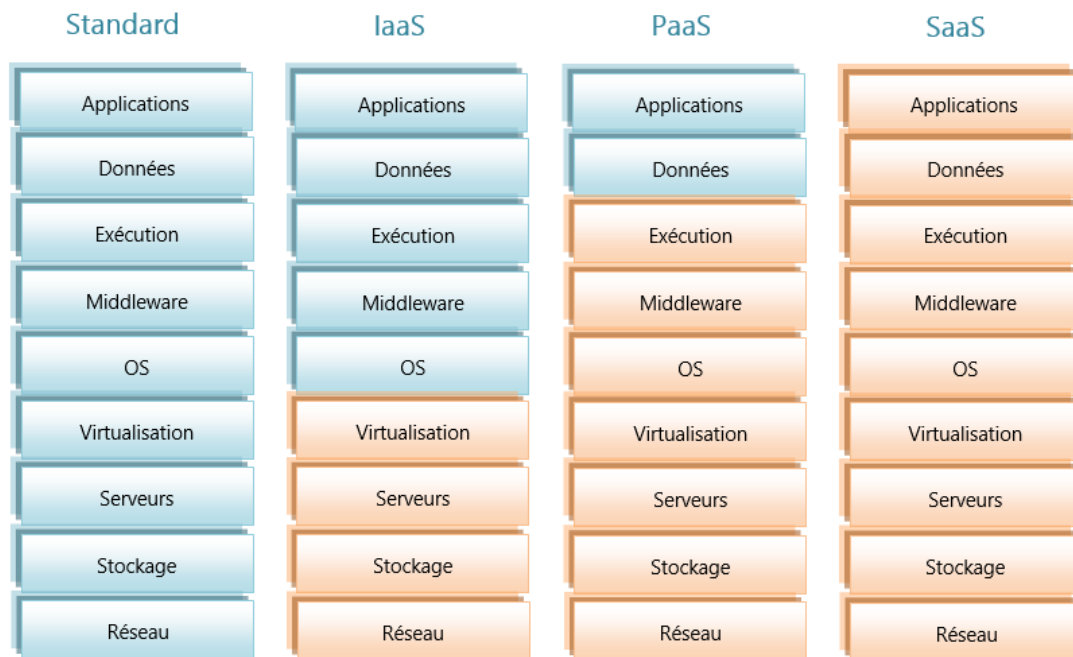


Figure 2.2: Les modèles de services Cloud [6]

2.3.2 Modèles de déploiement

- **Cloud public** : Un ensemble de services et de ressources disponibles à la demande, accessibles via Internet et gérés par un prestataire externe, ces ressources sont partagées entre plusieurs clients [28].
- **Cloud privé** : Un ensemble de services et de ressources disponibles pour un seul client. Ce modèle peut être créé et géré par l'entreprise elle-même ou par un prestataire externe [28].

- **Cloud hybride** : Une composition des deux modèles précédents [28].

2.3.3 Architecture orientée services (SOA)

Actuellement, l'Internet des Objets (IoT) connaît une expansion rapide. Son utilisation dans une multitude de secteurs visant à simplifier la vie des individus, pose toutefois un défi considérable qui est de relier le monde physique au monde numérique. Dans cette optique, l'architecture orientée services (SOA) offre une solution en facilitant la connexion et l'intégration des appareils IoT [30].

L'architecture orientée services est un style architectural qui utilise des services pour créer des systèmes complexes où les applications autonomes peuvent se connecter entre elles. Dans les systèmes IoT basés sur SOA, chaque appareil peut à la fois consommer et fournir des services via des API compatibles, permettant une communication et une utilisation multiples à travers le système. De plus, les systèmes SOA présentent une flexibilité et adaptabilité des services, permettant une interopérabilité entre des services hétérogènes sans nécessiter de modifications [31].

2.4 Définition du problème

Pour pallier l'absence d'un service IoT spécifique dans le cloud répondant à une requête envoyée par un client, une composition de services sélectionnés à partir d'un environnement multicloud est proposée. Cette solution offre une fonctionnalité similaire à celle du service recherché. Cependant, les besoins des clients peuvent être contradictoires, par exemple, minimiser les coûts tout en réduisant le temps de réponse, ce qui nécessite de trouver un équilibre entre différents objectifs, impliquant ainsi une optimisation multi-objectifs. De plus, les résultats obtenus sont influencés par une diversité de préférences des utilisateurs. Pour illustrer, l'utilisateur peut accorder plus d'importance à la fiabilité du service plutôt qu'à sa rapidité. Ce cas soulève un défi supplémentaire : sélectionner une composition de services en fonction des préférences spécifiques de chaque utilisateur afin de fournir une solution adaptée à leurs besoins uniques.

2.5 Approches de l'optimisation multi-objective

Il convient de noter que les travaux sur la sélection des services IoT dans le cloud sont en cours de développement. Parmi les approches identifiées, on peut citer les suivantes :

2.5.1 Optimisation via les métaheuristiques

Les métaheuristiques sont des algorithmes d'optimisation conçus pour résoudre des problèmes complexes qui ne peuvent être efficacement traités par des approches classiques. Inspirés par des processus naturels tels que la génétique ou le comportement des essaims, ces méthodes explorent un vaste espace de recherche pour trouver la meilleure solution possible [32].

Pour surmonter la complexité du problème proposé, plusieurs méthodes métaheuristiques ont été suggérées. Comme indiqué dans cet article [8], l'auteur a mené des expériences sur le même cas d'étude que le nôtre. Dans les sections suivantes, nous détaillerons le fonctionnement de quelques-unes des métaheuristiques les plus répandues.

2.5.1.1 Algorithme Génétique de Tri non Dominé II (NSGA-II)

L'algorithme NSGA-II est inspiré du processus d'évolution naturelle. basés sur les principes de la survie du plus apte, de la reproduction et de la mutation [32].

Principes de fonctionnement de l'algorithme :

1. Initialisation :

- Génération d'une population parente P de taille N , représente un ensemble de compositions initiales générées aléatoirement [7].
- Tri de P selon le concept de non-dominance, attribuant à chaque solution une valeur de fitness qui reflète son niveau de non-dominance [7].

2. Création de la population de descendants Q :

- Application des opérations de sélection par reproduction, recombinaison et mutation sur P pour obtenir Q de taille N .

3. Formation de la population R_t :

- Fusion de P et Q pour former R_t de taille $2N$.
- Tri de R_t selon le concept de non-dominance [7], tel que présenté dans la figure ci-dessus.

4. Sélection des meilleures solutions :

- Sélection des solutions de R_t appartenant aux frontières de Pareto $F_1, F_2, \dots$ jusqu'à ce que la taille de la population atteigne N .
- Pour les solutions de la dernière frontière sélectionnée, utilisation d'un algorithme de tri garantissant la sélection des individus les moins encombrés afin de maintenir la diversité parmi les solutions choisies [7].

5. Critère d'arrêt :

- Répétition du processus jusqu'à ce qu'un critère d'arrêt soit atteint, tel qu'un nombre maximal d'itérations ou une convergence suffisante des solutions non dominées, l'algorithme aura identifié les compositions de services optimales [7].

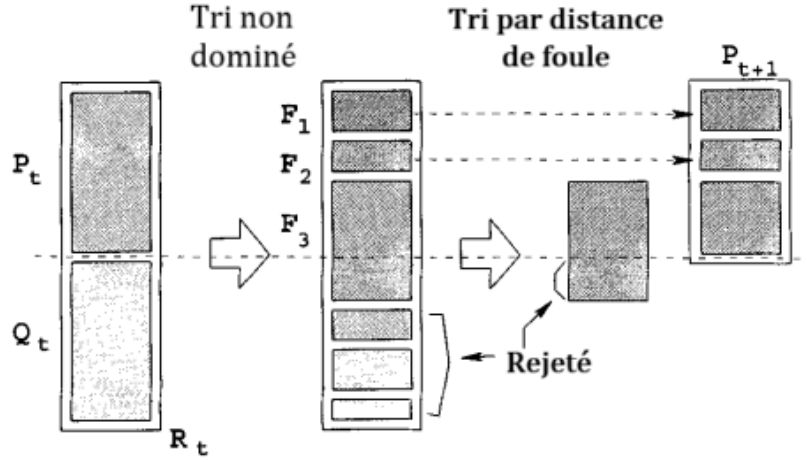


Figure 2.3: Procédure de l'algorithme NSGA-II [7]

2.5.1.2 Algorithme Génétique de Tri non Dominé III (NSGA III)

L'algorithme NSGA-III, représente une évolution de l'algorithme NSGA-II, spécialement conçue pour résoudre les problèmes d'optimisation avec plus de deux objectifs. Bien qu'il repose sur le même principe que NSGA-II, la distinction majeure entre NSGA-II et NSGA-III réside dans le fait que ce dernier abandonne la distance de foule pour la sélection des solutions basées sur leur fitness F_t au lieu de cela, il normalise les fonctions objectifs et les associe individuellement à un point de référence utilisé pour exprimer les préférences. Chaque point représente une direction dans l'espace des objectifs. [13].

2.5.1.3 Algorithme Génétique de Tri non Dominé III (RNSGA-III)

RNSGA-III est une amélioration de l'algorithme NSGA-III, intégrant une nouvelle méthode de sélection des points de référence dans le processus d'optimisation. Ces points de référence, correspondant à des valeurs utilisées pour orienter l'algorithme vers certaines solutions préférées, sont sélectionnés selon une approche basée sur des lignes de référence pour connecter ces points de référence à un point central dans un espace multidimensionnel. À la fin, les solutions sont évaluées par rapport à ces points de référence, permettant ainsi d'identifier les solutions optimales pour chaque objectif. Ce processus garantit l'obtention d'une variété de solutions qui correspondent de manière plus équilibrée aux points de référence. [8]

2.5.1.4 Optimisation par Essaim de particules Multi-Objectifs (OMOPSO)

C'est une variante de l'algorithme d'optimisation par essaims de particules (PSO), inspiré du comportement des oiseaux qui se déplacent en groupe, appelé essaim, mais adaptée pour résoudre des problèmes d'optimisation multi-objectifs.

Principes de fonctionnement

- Chaque particule représente une solution candidate au problème d'optimisation. Les particules se déplacent dans l'espace de recherche en fonction de leur propre expérience et de celle de leurs voisines.
- L'algorithme utilise la domination de Pareto pour évaluer les solutions. Les leaders sont sélectionnés par un tournoi binaire basé sur leur valeur d'encombrement. [33]
- Deux archives externes sont utilisées : une pour stocker les leaders actuels et une autre pour stocker les solutions finales. [34]
- Facteur d'encombrement est utilisé pour filtrer les leaders dans l'archive externe, seuls les leaders avec les meilleures valeurs d'encombrement sont conservés. [34]
- Deux opérateurs de mutation sont utilisés, la mutation uniforme qui est la plage de variabilité autorisée pour chaque variable de décision reste constante au fil des générations et la mutation non uniforme la plage de variabilité autorisée pour chaque variable de décision diminue avec le temps). Ces derniers, modifient les valeurs des variables de décision d'une particule avec une certaine probabilité afin d'explorer et exploiter l'espace de recherche. [34]
- Enfin le concept de dominance-epsilon est utilisé pour fixer la taille de l'archive externe, dépendant de la valeur epsilon définie par l'utilisateur. [34]

2.5.2 Optimisation via l'apprentissage par renforcement

Cet article [35] présente une approche de sélection de composition de services cloud dans un environnement multi-cloud reposant sur un système multi-agents. Nous allons expliquer en détail cette approche, en mettant particulièrement en lumière l'algorithme proposé.

2.5.2.1 Apprentissage par renforcement multi-agents

Ce travail a utilisé une nouvelle technique d'apprentissage par renforcement qui consiste à entraîner plusieurs agents coopérant pour trouver une politique optimale permettant de sélectionner les meilleurs services avec une meilleure QoS. Contrairement à l'apprentissage

par renforcement traditionnel, où un seul agent est impliqué dans le processus de décision Markovien (MDP).

2.5.2.2 Algorithme Deep Q-network (DQN)

Un réseau de neurones DQN est constitué de plusieurs couches, prenant en entrée l'état s et retournant en sortie les valeurs de la fonction état-action pour chaque action.

L'algorithme, utilise deux réseaux de neurones, un réseau primaire, qui est mis à jour à chaque interaction de l'agent avec son environnement, et un réseau cible, qui est mis à jour périodiquement. Le but principal de ce dernier est de fournir des cibles stables pour le réseau primaire afin qu'il puisse apprendre de manière plus efficace, ce qui stabilise l'apprentissage et accélère le processus. De plus, l'utilisation d'une mémoire de relecture (replay memory) dans laquelle l'agent stocke ses expériences et les utilise ultérieurement pour apprendre de ses expériences passées est également une composante essentielle de l'algorithme [36].

2.5.2.3 Limitations de cette approche

- Cette approche repose sur une fonction de scalarisation à utilité linéaire $f: \mathbb{R}^d \rightarrow \mathbb{R}$, où d est le nombre d'objectifs, dans le but de transformer un problème multi-objectifs en un problème à objectif unique. Toutefois, des études ont démontré que cette approche n'est pas toujours appropriée. Cette simplification peut s'avérer impossible, irréalisable ou indésirable [37]. De plus, les préférences de l'utilisateur doivent être établies dès le début pour pouvoir utiliser la fonction de scalarisation, ce qui n'est pas toujours réalisable étant donné la variabilité entre les individus.
- les travaux existants ont tendance à négliger l'incertitude liée à la partialité d'observation et à la grande dimension de l'espace d'action dans le contexte multi-agent, conduisant ainsi à l'adoption de politiques sous-optimales. [38].

2.6 Comparaison des approches d'optimisation

Ci-dessous, Nous présentons un tableau récapitulatif de l'état de l'art des algorithmes pour l'optimisation de la sélection des services IoT dans le cloud. Ce tableau met en évidence leurs limitations et leurs avantages respectifs, facilitant ainsi une comparaison visuelle.

Algorithme	Avantages	Limitations
NSGA-II	• Capable de maintenir une population diversifiée de solutions. • Adaptabilité à différentes configurations de problèmes. • Facilité d'implantation.	• Inefficace pour résoudre des problèmes avec plus de deux objectifs [13]. • Non conçu pour traiter les contraintes [39]. • Sensible aux réglages des paramètres. • Temps de réponse relativement lent [40].
NSGA-III	• Les mêmes avantages que ceux du NSGA-II.	• Le processus de normalisation impacte le comportement de l'algorithme [41].
RNSGA-III	• Focalisation sur les préférences de l'utilisateur [8]. • Remplacement de la distance de foule de NSGA-III par des lignes de référence pour maintenir la diversité des solutions [8].	• Utilisation inefficace des ressources de calcul en raison de points de référence qui ne contribuent pas de manière significative à la résolution du problème [8].
OMOPSO	• L'approche d'ϵ-dominance encourage la diversité des solutions. • L'archive de solutions leaders guide l'essaim vers des régions prometteuses du Pareto.	• Sensible aux réglages des paramètres [42].
MA + DQN	• Utilisation d'une nouvelle technique prometteuse en RL.	• La transformation d'un problème multi-objectifs en mono-objectif est déconseillée [37]. • Les préférences de l'utilisateur doivent demeurer constantes et uniformes pour l'ensemble des utilisateurs.

Tableau 2.1: Avantages et limitations des différentes approches de sélection de services IoT dans le cloud.

2.7 Conclusion

Dans ce chapitre, nous avons introduit les différentes notions des systèmes IoT et du cloud, puis nous avons examiné les avantages et les limitations des différentes approches proposées pour la sélection de composition de services, dont les métaheuristiques et l'apprentissage par renforcement avec objectif unique via l'algorithme DQN et un environnement avec agents multiples.

À notre connaissance, notre problème n'a jamais été résolu avec une optimisation multi-objectifs par renforcement. La plupart des approches sont basées sur des métaheuristiques, bien que cette dernière approche ait prouvé son efficacité dans divers domaines de recherche. C'est pourquoi nous nous intéressons à ouvrir la voie dans ce domaine d'actualité afin de déterminer son utilité potentielle.

Dans le prochain chapitre, nous présenterons les détails de conception de notre environnement, ainsi que les algorithmes choisis pour l'implémentation.

Chapitre 3

Conception

3.1 Introduction

Après avoir exploré les notions de base liées à notre système et étudié les systèmes existants dans la littérature, nous entamons désormais la phase de conception de notre solution. Cette étape revêt une importance capitale dans tout projet, car elle permet de concrétiser les idées sous forme d’une architecture fonctionnelle. Nous abordons ce processus en détaillant la conception de notre environnement et en expliquant les algorithmes d’apprentissage automatique et profond à objectifs multiples.

3.2 Architecture globale du système

Pour résoudre le problème de sélection des services IoT dans un environnement multi-cloud, deux processus sont mis en place. Tout d’abord, le processus de composition est engagé. À ce stade, des compositions de services approximativement équivalentes en termes de fonctionnalités sont définies pour les services que le client peut demander dans une requête.

Cependant, notre intérêt ne se porte pas sur ce premier processus. Il débute plutôt à partir de l’étape de sélection de la composition déjà définie. Nous considérons donc que les services sont déjà composés.

Pour le processus de sélection, lorsque qu’un client soumet une requête, un serveur vérifie initialement sa disponibilité. Si elle n’est pas disponible, le processus passe alors à l’étape de sélection de la composition équivalente. Les services de la composition sont disponibles sur plusieurs clouds avec des qualités de service (QoS) différentes. C’est à ce stade que l’optimisation par renforcement multi-objectifs (MORL) intervient. Cet algorithme détermine de quel cloud récupérer les différents services de la composition afin de

l'optimiser.

Le schéma ci-dessous illustre clairement ce processus :

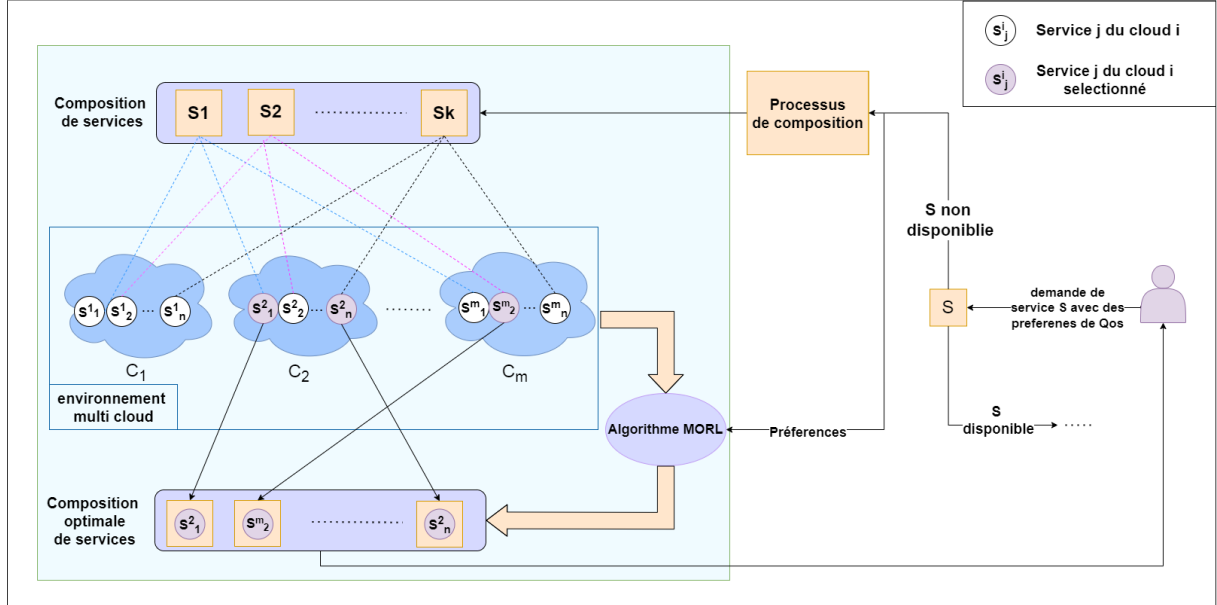


Figure 3.1: Architecture de composition et sélection des services dans le multi-cloud [8]

3.3 Formulation du problème

3.3.1 Codage de la solution

On considère la composition de services comme un vecteur où chaque élément représente un service. Pour chaque élément, il existe un ensemble de score de QoS, également représenté par un vecteur, dont les éléments sont respectivement calculé à partir des QoS la consommation d'énergie, le coût, le temps de réponse et le nombre de clouds. La qualité de service de cette composition est déterminée par la somme des vecteurs de ses composants individuels, élément par élément. Ainsi, notre objectif est de sélectionner les services dans ce vecteur de manière à maximiser individuellement chaque élément du vecteur de ses scores de QoS, d'où la nécessité d'une optimisation multi-objectifs.

Une solution (service) dans la composition est représentée comme suit :

s^j_i : le service dont l'identifiant est i qui se trouve dans le cloud j

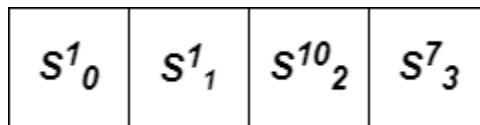


Figure 3.2: Exemple d'une composition de quatre services

Dans cette composition, les services dont l'identifiant est 0 et 1 sont pris du cloud 1, le service 2 du cloud 10 et le service 3 du cloud 7.

3.3.2 Prétraitement des données

Le prétraitement des données est une étape indispensable qui permet à l'agent d'apprendre de manière efficace. Si les données ne sont pas correctement traitées, l'agent peut diverger et ne pas apprendre correctement. Dans notre dataset, nous avons pris en compte les QoS, la consommation d'énergie, le coût, le temps de réponse et le nombre de clouds dans la composition.

Nous avons commencé par normaliser ces données pour qu'elles soient comprises entre 0 et 1. Ensuite, nous les avons multipliées par -1, car il s'agit d'objectifs à minimiser : une récompense élevée doit être obtenue lorsque ces QoS sont faibles.

$$x' = -\frac{x}{x_{\max}} \quad (3.1)$$

Cependant, nous avons rencontré un problème de divergence. Après investigation, nous avons découvert que cela était dû au fait que l'agent recevait très souvent des récompenses négatives. Pour résoudre ce problème, nous avons ajusté les données de telle sorte que les données soient comprises entre 0 et 1 comme suit :

$$x' = \left(-\frac{x}{x_{\max}}\right) + 1 + \frac{x_{\min}}{x_{\max}} \quad (3.2)$$

3.3.3 Conception de l'environnement

La conception de l'environnement joue un rôle essentiel dans le processus d'apprentissage par renforcement, influençant directement la politique d'apprentissage de l'agent tout en visant à améliorer ses performances.

Dans le contexte de ce projet, l'agent évolue au sein du vecteur des services. Il ajuste les éléments du vecteur de composition un par un tout en récupérant les services depuis différents clouds. Lorsqu'il atteint la fin de ce vecteur, il parvient à déterminer la combinaison optimale en fonction des préférences de l'utilisateur, comme illustré dans la figure 3.4.

L'espace d'états : Chaque service disponible dans le multi-cloud est un état dans l'environnement. Le nombre total d'états est calculé en multipliant le nombre de clouds dans le multi-cloud par le nombre de services dans la composition.

L'espace d'actions : Se compose de trois actions :

— Action 0 : Augmenter l'indice du cloud du service courant de 1, *i.e.* $s_i^j \leftarrow s_i^{j+1}$.

- Action 1 : Augmenter l'indice du cloud du service courant de 5, *i.e.* $s_i^j \leftarrow s_i^{j+5}$.
- Action 2 : Passer au service suivant dans la composition, *i.e.* $s_i^j \leftarrow s_{i+1}^j$.

L'espace de récompenses : Le vecteur de récompenses $\vec{r}_i^j$ contient les scores de chaque QoS du service S_i^j , les valeurs des QoS sont normalisés, comprises entre 0 et 1.

$$\vec{r}_i^j = \vec{X}_i^j \quad \text{où} \quad \vec{X}_i^j \text{ contient les valeurs normalisées et traitées des QoS du service } S_i^j \quad (3.3)$$

Cette récompense n'est attribuée à l'agent qu'après avoir choisi l'action 2. Ainsi, une fois le service sélectionné, il peut obtenir la récompense correspondante.

L'état initial : défini comme étant le premier service de la composition, localisé dans le premier cloud où ce service est disponible.

L'état final : L'agent atteint cet état après avoir terminé la sélection du dernier service de la composition en choisissant l'action 2.

Ci-dessous, une figure illustre l'interaction de l'agent avec son environnement, depuis l'état initial où tous les services sont initialement hébergés dans le cloud 0, jusqu'à l'état final où l'agent a sélectionné les clouds où chaque service de la composition optimale est hébergé.

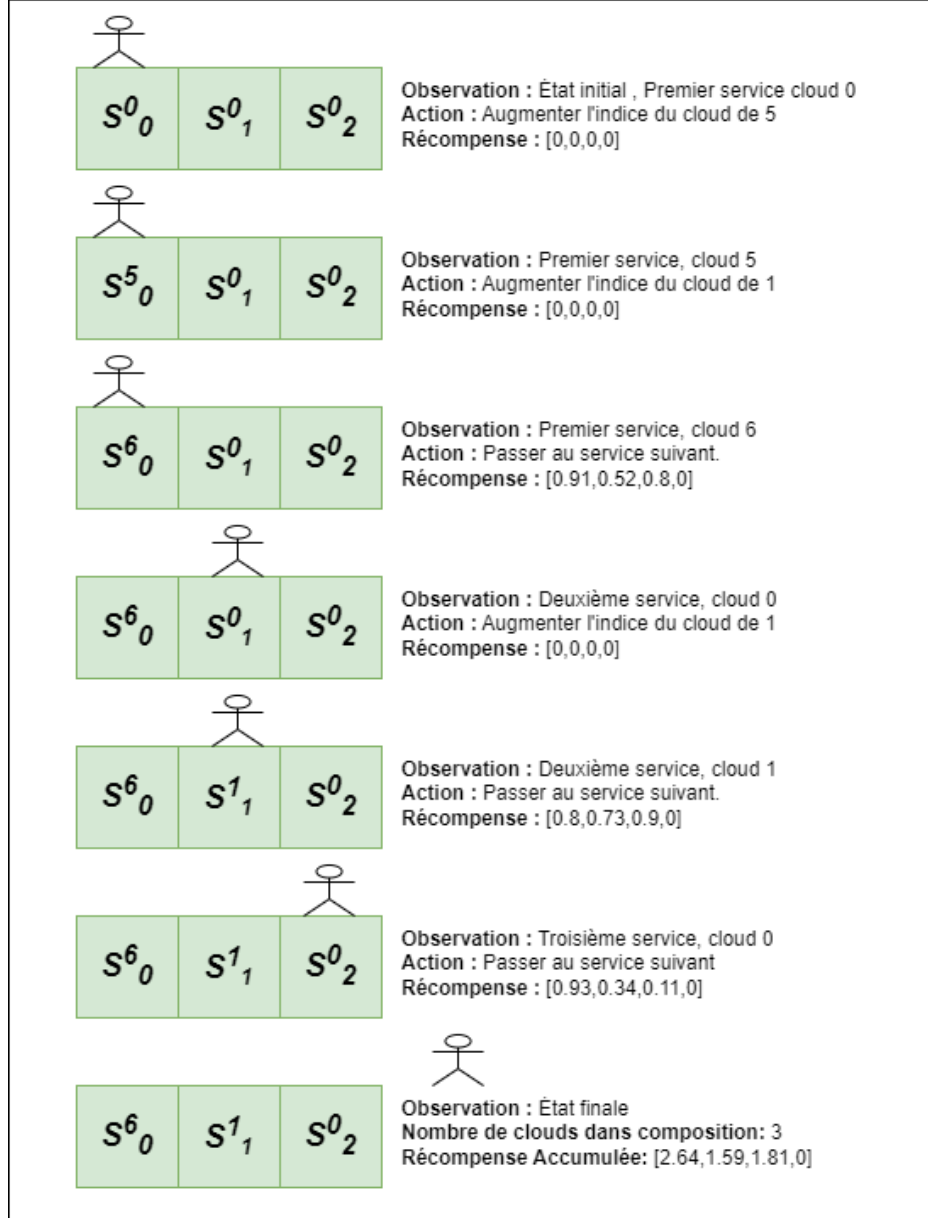


Figure 3.3: Exemple illustrant l'interaction de l'agent avec son environnement.

3.4 Algorithme multi-objetifs à politiques multiples par Q-Learning (MPMOQL)

3.4.1 Algorithme Q-Learning

Le Q-Learning [43] est un algorithme d'apprentissage par renforcement mono-objectif qui apprend directement des expériences acquises en interagissant avec l'environnement par le processus de prise de décision markovienne (MDP). Il évalue l'intérêt des actions dans différents états par une table de valeurs, appelée la Q-table mise à jour au fur et à mesure ce qui lui permet de s'améliorer continuellement.

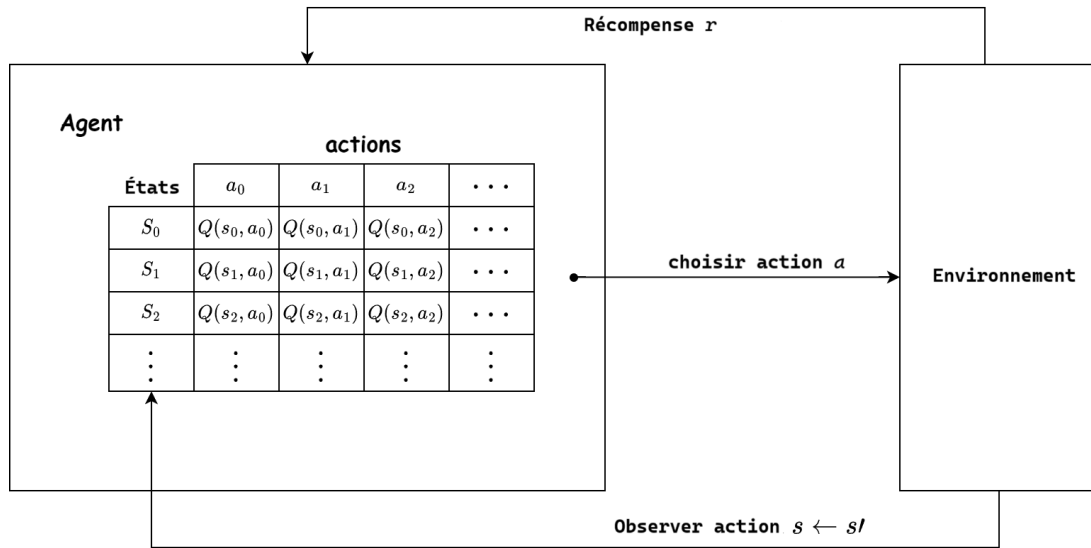


Figure 3.4: Table de Q-valeurs dans l'algorithme Q-Learning [9]

3.4.2 Algorithme MPMOQL

En apprentissage mono-objectif, la table du Q-learning est utilisée pour stocker la récompense attendue pour la combinaison de l'état s et de l'action a , donc $\hat{Q}(s, a)$. Dans un cadre multi-objectifs, la Q table est étendue pour incorporer les objectifs, c'est-à-dire $\hat{Q}(s, a, o)$. Ainsi, les récompenses attendues pour chaque état, action et objectif peuvent être stockées, récupérées et mises à jour séparément.

MPMOQL est un algorithme multi-politique (Multi-policy) ce qui veut dire que l'agent apprend un ensemble de politiques, chacune optimisée pour une préférence particulière. Ces politiques peuvent ensuite être utilisées pour résoudre des problèmes avec de nouvelles préférences. Ci-dessous se trouve l'algorithme expliquant le fonctionnement de MPMOQL.

Algorithme 3.1 : Algorithme Q-Learning Multi-Objectifs

Initialiser la table Q-valeur $\hat{Q}(s, a, o)$ avec des valeurs aléatoires.

Pour tout épisode T Initialiser l'état courant s .

Repeter

Choisir une action a dans l'état s en utilisant la stratégie epsilon-greedy.

Exécuter l'action a et observer le nouvel état $s' \in S$ et le vecteur de récompense

$\vec{r} \in \mathbb{R}$.

Pour Tout objective $o \in O$

$\hat{Q}(s, a, o) \leftarrow \hat{Q}(s, a, o) + \alpha[\vec{r}(s, a, o) + \gamma\hat{Q}(s', greedy_{a'}(s'), o) - \hat{Q}(s, a, o)];$

Fin

Mettre à jour l'état courant : $s \leftarrow s'$.

Jusqu'à ce que s soit terminal

Fin

3.5 Algorithme Réseaux conditionnés par Pareto (PCN)

3.5.1 Définition

PCN est un algorithme d'apprentissage profond par renforcement (DRL) qui fonctionne efficacement avec un grand nombre d'objectifs [44].

PCN se distingue des autres algorithmes de DRL car il vise à guider l'agent vers une récompense spécifique définie comme paramètre lors de son initialisation, plutôt que de maximiser les récompenses, PCN cherche à apprendre à l'agent les décisions séquentielles qui lui permettent d'obtenir une récompense cumulée définie.

L'idée principale derrière PCN est d'intégrer la technique d'apprentissage supervisé pour apprendre les politiques en transformant ainsi le problème en un problème typique de classification [44]. Il utilise un seul réseau neuronal qui prend en entrée un tuple $\langle s, \hat{h}, \hat{R} \rangle$ où s est l'état, $\hat{R}$ est la récompense cumulative que l'algorithme doit atteindre et $\hat{h}$ est l'horizon qui définit combien d'intervalles de temps l'agent doit parcourir afin d'atteindre la récompense accumulée R . La sortie de ce réseau de neurones est l'action $a \in A$. L'algorithme doit donc prédire quelle action lui apportera la récompense $\hat{R}$ dans un horizon $\hat{h}$ à partir d'un état s .

3.5.2 Dataset des Expériences de l'Agent

Au début, le dataset est vide car l'agent n'a pas encore collecté d'expérience. Il commence donc par prendre des actions aléatoires afin d'observer son environnement et d'ac-

quérir de nouvelles expériences qu'il va par la suite stocker dans le dataset. On ajoute un point de données étiqueté $\langle s, \hat{h}, \hat{R} \rangle = \langle s_t, h_t, R_t \rangle$ et en sortie $a = a_t$ au dataset.

3.5.3 Entraînement de l'algorithme

Le réseau de neurones du PCN [44] prend en entrée l'état s , l'horizon h , et la récompense accumulée R que l'on souhaite déterminer. Un embedding est utilisé pour réduire la dimensionnalité : un pour l'observation et un autre pour l'horizon et la récompense accumulés. Ces derniers sont ensuite multipliés par un facteur d'échelle pour normaliser les entrées du réseau neuronal, puis passés à travers une seule couche cachée entièrement connectée avec une activation ReLU, suivie d'une sortie avec une activation sigmoïde pour la classification.

La sortie réelle y est comparée à la sortie prédite $\hat{y} = \pi(s_t, h_t, R_t)$. La politique est ensuite mis à jour en utilisant la fonction de perte de la cross-entropie [44] :

$$H = - \sum_{a \in A} y_a \log \pi(a | s_t, h_t, R_t) \quad \text{avec } y_a = \begin{cases} 1 & \text{si } a = a_t \\ 0 & \text{sinon} \end{cases} \quad (3.4)$$

Le réseau est réentraîné périodiquement selon un hyperparamètre en s'assurant que de nouvelles expériences ont été collectées et que le dataset a été suffisamment amélioré.

3.6 Conclusion

Dans ce chapitre, nous avons exposé l'étude conceptuelle de notre travail, détaillant la méthodologie employée pour mettre en œuvre notre travail. Nous avons débuté par la conception de la structure de l'environnement englobant la définition des espaces d'état, d'action et des fonctions de récompense. Par la suite, nous avons mis en pratique l'algorithme MPMOQL et PCN.

Dans le chapitre suivant, nous nous concentrerons sur l'implémentation concrète de notre solution. Nous fournirons des détails techniques sur les différentes étapes de l'implémentation, en expliquant les outils que nous avons utilisés et les choix que nous avons faits. Ce chapitre fournira une vue d'ensemble complète du fonctionnement pratique de notre système de sélection des services IoT du cloud.

Chapitre 4

Implémentation de la solution

4.1 Introduction

Après avoir présenté la conception de notre système dans le chapitre précédent, ce chapitre se concentre sur l'implémentation et l'exécution de notre solution. Nous parlerons de l'environnement matériel sur lequel notre solution a été développée, ainsi que les langages de programmation et les outils utilisés. De plus, nous discuterons des métriques utilisées pour évaluer notre solution. Enfin, nous présenterons les interfaces graphiques, suivies d'une illustration détaillée de notre système proposé.

4.2 Environnement de travail

4.2.1 Environnement matériel :

Pour le matériel de la machine, nous avons opté pour un ordinateur avec les caractéristiques suivantes :

- HP ELITEBOOK.
- Processeur Intel(R) Core(TM) i7-8550U CPU @ 1.80GHz 1.99 GHz.
- Ram 8,00 Go.
- Système d'exploitation Windows 11 de type 64 bits.

4.2.2 Environnement logiciel

Language de programmation

- **python** : Langage de programmation puissant et opensource, avec des structures de données avancées et une approche simple mais efficace de la programmation orientée

objet, et qui est vastement utilisé dans divers domaines et sur différentes plateformes.

- **MATLAB** : Conçu par MathWorks, est un langage de programmation et un environnement de calcul numérique offrant des fonctionnalités telles que le traçage de graphiques et l'implémentation d'algorithmes [45]. Nous l'avons utilisé pour tracer les graphes de nuage de points en paire (*voir annexes A et B*).

Bibliothèques utilisées

- **MO-Gymnasium** : Bibliothèque Python ouverte qui propose une API standard et facile pour la conception d'environnements capable de représenter des problèmes d'apprentissage par renforcement, Ces environnements utilisent essentiellement l'API standard de Gymnasium, à l'exception que les récompenses renvoyées par l'environnement sont sous forme de tableaux numpy [46].
- **MORL-baselines** : Bibliothèque d'algorithmes d'apprentissage multi-objectifs par renforcement (MORL), elle contient des implémentations d'algorithmes MORL et suit précisément l'API de MO-Gymnasium [47].
- **Weights and Biases (WandB)** : Plateforme pour développeurs d'IA, avec des outils pour l'entraînement des modèles, le réglage fin des modèles et l'exploitation des modèles de fondation [48].
- **Pymoo** : Propose des algorithmes métaheuristiques pour l'optimisation mono et multi-objectifs, ainsi que de nombreuses autres fonctionnalités telles que la visualisation et l'aide à la prise de décision [49].

Outils utilisés pour la conception de l'interface

- **HTML** : Le langage de balisage Hypertexte (HTML) est un langage standard pour les documents conçus pour être affichés dans un navigateur web. Il définit le contenu et la structure de la page web [50].
- **CSS** : Les feuilles de style en cascade (CSS) est utilisé pour spécifier la présentation et le style d'un document écrit dans un langage de balisage tel que HTML [51].
- **Java Script** : Un langage de programmation centrale du Web. Il donne aux développeurs la possibilité de créer des pages web interactives et dynamiques, permettant ainsi aux visiteurs d'interagir avec le contenu et d'exécuter des actions complexes [52].
- **Django** : Un framework web basé sur Python, gratuit et open-source, qui s'exécute sur un serveur web. Il suit le modèle architectural model–template–views et qui encourage le développement rapide et une conception propre et pragmatique [53].

4.3 Dataset

Ce dataset utilisé par A.Zebouchi et al. dans son travail référencé dans l'article [8], contient les valeurs de qualité de service (QoS) des services disponibles dans le cloud du multicloud. Il se compose de 12 dossiers, chacun représentant un multicloud différent, avec des nombres de services et de clouds variés. Nous avons choisi de travailler avec le dossier contenant le multicloud comprenant 50 services et 20 clouds. Chaque service est caractérisé par sa consommation d'énergie, son coût, sa fiabilité, sa disponibilité et son temps de réponse en termes de QoS. Le dataset contient également les services du plan de composition de chaque cloud. Ci-dessous, nous présentons un échantillon du dataset accompagné de quelques informations sur la répartition des QoS des services.

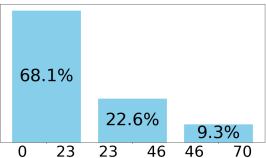
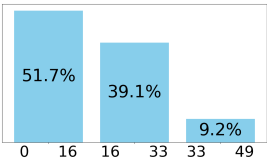
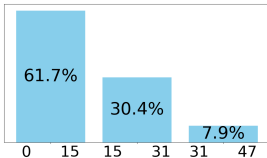
		Coût	Consommation d'énergie	Temps de réponse
Cloud	Service			
0	0	9.78	12.47	15.96
	1	13.22	17.00	24.452
	...			
	49	6.47	12.41	23.27
...				
19	0	10.009	4.209	16.721
	...			
	49	17.275	4.799	3.037

Tableau 4.1: Exemple de données de quelques services dans le multi-cloud à améliorer.

4.4 Implémentation

L'implémentation de ce projet se déroule en deux étapes distinctes. Tout d'abord, nous devons construire l'environnement tel qu'il est décrit dans le chapitre 3, en utilisant l'API mo-gymnasium. Ensuite, nous passerons à l'utilisation de MORL-baselines pour appliquer les algorithmes multi-objectifs par renforcement. Dans cette section, nous présenterons de

manière détaillée les hyperparamètres que nous avons sélectionnés après avoir effectué plusieurs tests.

4.4.1 Hyperparamètres

Pour garantir la convergence de l'agent vers la solution optimale, il est essentiel de sélectionner les bons hyperparamètres parmi ceux que nous avons utilisés pour ajuster les algorithmes MPMOQL et PCN. Voici les hyperparamètres en question :

Hyperparamètres communs entre les deux algorithmes

- **Taux d'apprentissage** : Contrôle la taille des ajustements des paramètres de la politique ou de la fonction de valeur d'un agent. Un taux élevé accélère la vitesse de convergence mais peut causer une instabilité, tandis qu'un taux bas favorise la stabilité au détriment de la vitesse de convergence [54].
- **Facteur de remise** : Équilibre l'importance des récompenses immédiates par rapport aux récompenses futures.
- **Pas de temps total** : Le nombre de pas de temps représente le total des pas de temps dans tous les épisodes.

Hyperparamètres de MPMOQL

- **Étapes de décroissance d'epsilon** : Employé dans les stratégies d'epsilon-greedy. Il indique combien d'étapes sont nécessaires pour réduire progressivement la valeur d'epsilon au cours de l'entraînement.
- **Pas de temps par itération** : Indique combien de pas de temps l'agent prend pour interagir avec l'environnement avant de mettre à jour sa stratégie ou sa fonction de valeur.

Hyperparamètres de PCN

- **Taille du lot (Batch size)** : Nombre d'échantillons traités ensemble avant la mise à jour des paramètres. Affecte la stabilité de l'apprentissage.
- **Taille maximale du tampon (max buffer size)** : La taille de la mémoire de relecture (replay memory) où l'agent stocke ses expériences, doit être équilibrée afin de disposer d'un nombre suffisant d'expériences pour un apprentissage efficace tout en conservant des ressources computationnelles gérables.
- **num_er_episodes** : Nombre d'épisodes pour remplir la mémoire de relecture.

- **num_model_updates** : Controle la fréquence à laquelle le réseau est ré-entraîné après un nombre défini d'épisodes.

Hyperparamètre	MPMOQL	PCN
Taux d'apprentissage	0.15	0.001
Étapes de décroissance d'épsilon	9000	-
Facteur de remise	1	1
Pas de temps par itération	1000	-
Pas de temps total	10000	10000
Taille du lot	-	256
Taille maximale du tampon	-	2000
num_er_episodes	-	50
num_model_updates	-	100

Tableau 4.2: Hyperparamètres attribués pour les algorithmes MPMOQL et PCN

4.5 Indicateurs de performances

Les indicateurs de performance nous fournissent une méthode efficace et fiable pour évaluer notre solution.

- **Temps d'exécution** : Mesure en secondes le temps nécessaire pour que l'algorithme termine son exécution.
- **Hypervolume (HV)** : Indicateur à maximiser, mesure le volume dominé par l'ensemble des solutions optimales trouvées par l'algorithme, par rapport à un point de référence fixe dans l'espace objectif, Ce point de référence est souvent établi sur des valeurs inférieures à celles de toute solution réalisable [8]. (*Cet article [55] fournit une explication détaillée de l'hypervolume*) .
- **Distance Générée Inversée (IGD)** : Indicateur à minimiser, calcule la distance moyenne de chaque point du front de Pareto à la solution la plus proche. Un faible IGD indique que l'algorithme converge bien vers les solutions de Pareto [8].
- **Cardinalité** : Indicateur à maximiser, détermine la diversité et la richesse des solutions, une cardinalité élevée implique que les solutions trouvées sont bien réparties dans l'espace des objectifs et ne sont pas regroupées ensemble.

4.6 Tests et analyse

Après avoir implémenté notre environnement ainsi que les algorithmes MPMOQL et PCN, nous procédons à l'évaluation des résultats obtenus.

Temps d'exécution

Les temps d'exécution des deux algorithmes diffèrent considérablement :

- **MPMOQL** : 43 secondes.
- **PCN** : 276 secondes (4min et 36s).

Nous remarquons que le temps d'exécution de l'algorithme PCN est nettement supérieur à celui du MPMOQL.

Métriques d'apprentissage

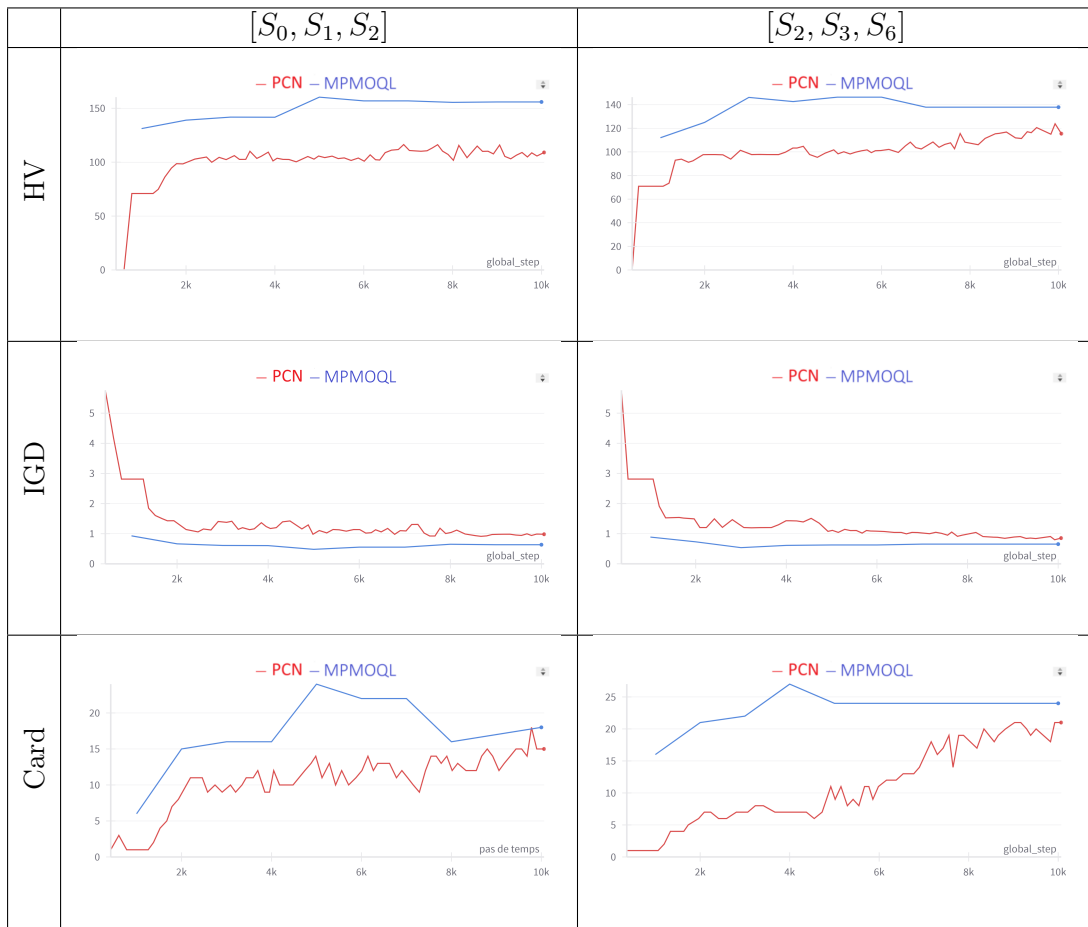


Tableau 4.3: Métriques obtenues lors de l'exécution des algorithmes MPMOQL et PCN.

Nous constatons que MPMOQL est meilleur que PCN lorsque l'on considère les trois indicateurs de performance suivants : l'hypervolume, l'IGD et la cardinalité.

Pour davantage de tests sur différentes compositions de services pour l'algorithme MPMOQL, veuillez vous référer à l'annexe B.

4.7 Discussion

L'algorithme MPMOQL a clairement produit de meilleures solutions, tant en termes de temps d'exécution que de résultats sur différentes métriques telles que l'hypervolume et l'IGD. Cela signifie que MPMOQL génère des résultats plus proches du vrai front de Pareto, tandis que la métrique de cardinalité montre que MPMOQL offre une plus grande diversité de solutions par rapport à PCN.

L'algorithme PCN prend beaucoup plus de temps d'exécution que MPMOQL. Cela s'explique par le fait que le premier est basé sur l'apprentissage profond, tandis que le second utilise uniquement l'apprentissage automatique. Le modèle profond est généralement plus complexe et utile pour résoudre des problèmes complexes.

PCN converge sans explorer l'espace de recherche davantage, ce qui peut entraîner une convergence précoce et un blocage sur un optimum local.

Pour certaines compositions de services, l'entraînement de l'agent avec l'algorithme MPMOQL peut entraîner le phénomène du "catastrophic forgetting", où l'agent oublie les expériences qu'il a apprises, ce phénomène se produit lorsque l'agent met à jour ses valeurs Q (Q -values) dans la Q -Table pendant l'entraînement, ce qui peut entraîner l'écrasement des connaissances acquises.

Cependant, les deux algorithmes n'ont pas produit de meilleurs résultats que ceux de NSGA-III (*voir annexe A pour les détails de l'implémentation de NSGA-III*). Plusieurs facteurs peuvent expliquer ces résultats :

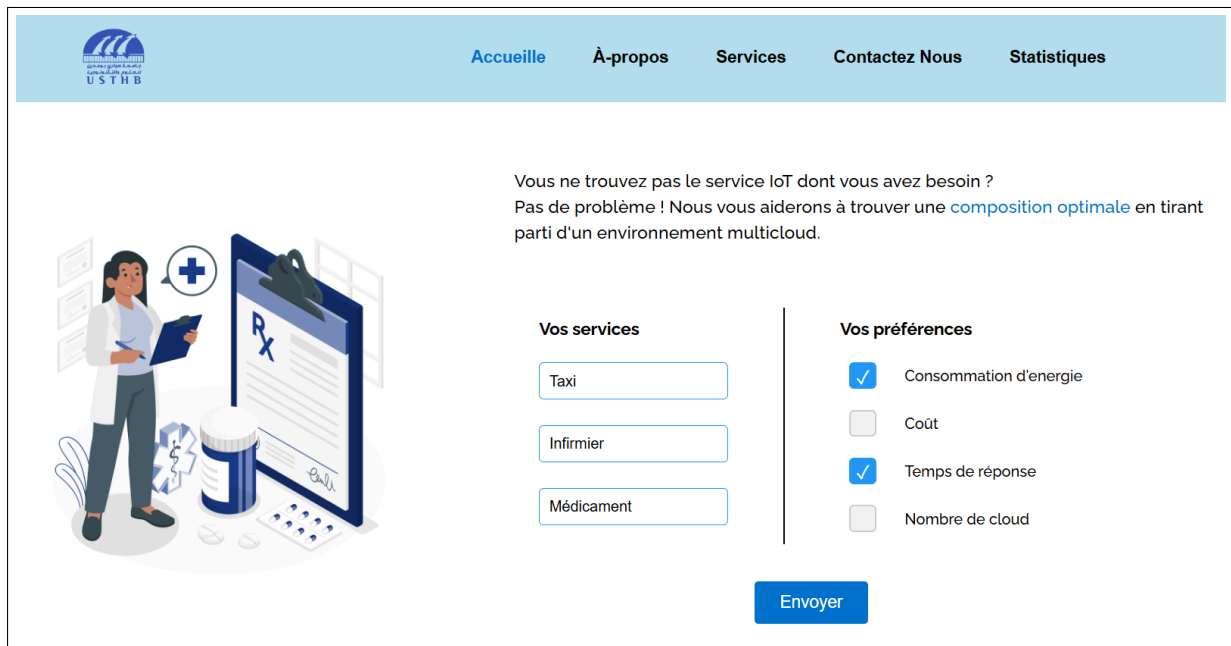
- L'apprentissage par renforcement est très sensible aux hyperparamètres. Avec un grand nombre d'hyperparamètres, leur réglage devient donc très complexe, surtout avec le manque de ressources matérielles, ce qui rend cette tâche encore plus difficile.
- La dynamique de l'environnement ou chaque composition de service souhaitée en entrée génère un environnement avec des récompenses différentes.
- Le manque d'exemples d'implémentation constitue un obstacle majeur. Les seuls exemples disponibles concernent principalement des jeux, ce qui est très éloigné de notre cas d'application.
- Le domaine de l'apprentissage par renforcement multi-objectifs en est encore à ses débuts, avec un manque de documentation et une communauté travaillant avec les MORL-baselines encore restreinte

4.8 Démonstration

Nous avons conçu une interface Web simple afin de pouvoir présenter notre travail de manière claire. Il est important de noter que cette interface ne représente pas le véri-

table processus de composition, car nous nous concentrons uniquement sur la sélection de services dans le cadre de notre projet.

Dans un premier temps, l'utilisateur entre les services qu'il souhaite inclure dans la composition et spécifie ses préférences en termes de QoS. Ensuite, il soumet ces informations, et l'algorithme se charge alors de trouver la composition optimale en back-end.



USTHB

Accueil À-propos Services Contactez Nous Statistiques

Vous ne trouvez pas le service IoT dont vous avez besoin ?
Pas de problème ! Nous vous aiderons à trouver une [composition optimale](#) en tirant parti d'un environnement multicloud.

Vos services

Taxi

Infirmier

Médicament

Vos préférences

☒ Consommation d'énergie

☐ Coût

☒ Temps de réponse

☐ Nombre de cloud

Envoyer

Figure 4.1: Interface conçue pour choisir les services de la composition

Par la suite, l'interface affiche à l'utilisateur les trois services de la composition, chacun avec ses QoS et l'identificateur du cloud d'où il a été récupéré.



Figure 4.2: Interface conçue pour afficher la composition sélectionnée

4.9 Conclusion

Au cours de ce chapitre, nous avons décrit le processus de mise en œuvre de la solution proposée pour notre problème, en détaillant l'environnement de développement, la description globale de l'implémentation, le dataset, les hyperparamètres et les métriques d'évaluation. Nous avons également réalisé une analyse globale des résultats obtenus par les deux algorithmes implémentés. Cette analyse a été suivie d'une démonstration d'un exemple conforme à notre démarche de conception établie dans le chapitre précédent.

Conclusion et perspectives

L'importance croissante de trouver un algorithme multi-objectifs d'optimisation pour la sélection des services IoT dans le cloud est due à l'augmentation rapide du nombre d'objets connectés. Cela fait que cette tâche est devenue cruciale, attirant de nombreuses recherches.

Dans ce travail, nous avons exploré les approches déjà proposées dans la littérature, majoritairement basées sur des algorithmes métaheuristiques, ainsi qu'une approche en apprentissage par renforcement mais dans un contexte mono-objectif. C'est donc ce qui a orienté notre choix vers le renforcement multi-objectifs. Par la suite, nous avons présenté la conception de notre environnement et le traitement de nos données afin de pouvoir aborder les algorithmes que nous avons appliqué. Enfin, nous avons mentionné notre environnement logiciel et matériel, puis présenté les résultats de notre exécution ainsi que l'interface utilisée pour les présenter.

Dans le système que nous avons proposé, nous avons démontré la faisabilité de l'implémentation de l'apprentissage par renforcement multi-objectifs dans notre cas d'étude. Pour ce faire, nous avons mis en œuvre deux algorithmes multi-objectifs : l'algorithme MOMPQL par apprentissage automatique, qui a fourni des résultats satisfaisants, et l'algorithme PCN par apprentissage profond. Cependant, l'algorithme PCN présente des limitations majeure : son temps d'exécution est considérablement plus long que celui de l'algorithme MOMPQL et il a tendance à converger prématurément sans explorer suffisamment l'espace de recherche.

Pour les travaux futurs, nous envisageons d'élargir la composition en augmentant le nombre de services et en incluant également plus de clouds, tout en prenant en considération des objectifs supplémentaires tels que la disponibilité, la fiabilité et le nombre de clouds dans le plan de composition. Ainsi, nous pensons qu'il serait pertinent d'appliquer le méta-apprentissage, un type d'apprentissage par renforcement qui peut mieux gérer la dynamique de l'environnement. Contrairement à l'apprentissage par renforcement classique, le méta-apprentissage ne sépare pas l'entraînement de la prédiction et utilise ce que l'on appelle l'apprentissage en ligne. Nous pensons que la combinaison de l'apprentissage

par renforcement multi-objectifs et du méta-apprentissage peut constituer une solution efficace pour la sélection des services IoT dans le cloud.

Bibliographie

- [1] Qing Cai, Ma Lijia, and Maoguo Gong. "a survey on network community detection based on evolutionary computation", *International Journal of Bio-Inspired Computation*, vol 08, num 02. 2014. <https://doi.org/10.1504/IJBIC.2016.076329>.
- [2] Binay Dhakal. Mdp (markov decision process) — rl (reinforcement learning) | by binay dhakal | medium. <https://medium.com/@binaydhakal35>, sep 2023. (Consulté le 09/05/2024).
- [3] Faris B. Misma. Structure of the neural network used for the deep q-learning network... | download scientific diagram. <https://www.researchgate.net>. (Consulté le 03/05/2024).
- [4] Sherif Mosad Abdelfattah , Kathryn Elizabeth Kasmarik, and Jiankun Hu. Evolving robust policy coverage sets in multi-objective markov decision processes through intrinsically motivated self-play. *Front. Neurorobot*, Volume 12 - 2018 . <https://doi.org/10.3389/fnbot.2018.00065>.
- [5] Architecture iot : L'essentiel à savoir | iot industriel blog. <https://iotindustriel.com/iot-iiot/architecture-iot-lessentiel-a-savoir/>.
- [6] Les types de services cloud | administration réseau | it-connect. <https://www.it-connect.fr/les-types-de-services-cloud/>. (Consulté le : 05/04/2024).
- [7] Kalyan Deb, Amrit Pratap, Sameer Agarwal, and T. A. M. T. Meyarivan. A fast and elitist multiobjective genetic algorithm : Nsga-ii | iee journals & magazine | iee explore, 2002. <https://doi.org/10.1109/4235.996017>.
- [8] Ahmed Zebouchi and Youcef Aklouf. prtmnsga-iii : a novel multi-objective algorithm for qos-aware multi-cloud iot service selection | annals of telecommunications. <https://doi.org/10.1007/s12243-023-01006-0>, Jan 2024.
- [9] godaehwa gominsu. Deep learning bible - 5. reinforcement learning - eng. <https://wikidocs.net/174536>. Visited on April 20, 2024.
- [10] Mathematical optimization - wikipedia. https://en.wikipedia.org/wiki/Mathematical_optimization. (Consulté le : 15 mai 2024).

- [11] Joyendu Bhadury H. A. Eiselt, Vladimir Marianov. *"Multicriteria Location Analysis"*. International Series in Operations Research Management Science.
- [12] Neeti Kashyap, Charan Kumari, and Rita Chhikara. Multi-objective optimization using nsga ii for service composition in iot, 2020. <https://doi.org/10.1016/j.procs.2020.03.214>.
- [13] Simon Wietheger and Benjamin Doerr. A mathematical runtime analysis of the non-dominated sorting genetic algorithm iii (nsga-iii). *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 2023.
- [14] Batta Mahesh. "machine learning algorithms - a review", *International Journal of Science and Research (IJSR)*, 2018. <https://doi.org/10.21275/ART20203995>.
- [15] Deep learning vs machine learning. Consulté le : 15 mai 2024.
- [16] Yuxi Li. Deep reinforcement learning : An overview. <https://arxiv.org/abs/1701.07274>, Nov 2018.
- [17] Bingjie Shen. Exploration methods in reinforcement learning. iee international conference on advances in electrical engineering and computer applications (aeeca). 2022. <https://doi.org/10.1109/AEECA55500.2022.9918998>.
- [18] Castronovo Michaël. Learning for exploration/exploitation in reinforcement learning. *IEEE International Conference on Advances in Electrical Engineering and Computer Applications (AEECA)*, 2012.
- [19] A generalized algorithm for multi-objective reinforcement learning and policy adaptation. *NeurIPS*. 2019. <https://doi.org/10.48550/arXiv.1908.08342>.
- [20] Zainab H. Ali, Hesham A.Ali, and Mahmoud M.Badawy. "internet of things (iot) : Definitions, challenges, and recent research directions". *International Journal of Computer Applications (0975 – 8887) Volume 128 – No.1*. (2015). <https://doi.org/10.5120/ijca2015906430>.
- [21] Hidayet Aksu. Researchgate. https://www.researchgate.net/figure/IoT-Architecture-Layers-and-Components_fig1_322975901.
- [22] Enying Gong Paul Scuffham Jaimon T Kelly, Katrina L Campbell. The internet of things : Impact and implications for health care delivery. *journal of medical internet reaserch*, 2020.
- [23] Ben Dhaou I. Rahmani M. A. Westerlund T. Al. Tuan Nguyen G., Mai A. "iot-based continuous glucose monitoring system : A feasibility study". *Procedia Computer Science, Volume 109, Pages 327-334* (2017). <https://doi.org/10.1016/j.procs.2017.05.359>.

- [24] Aparna Shashikant Joshi, Rishabh Cambo, Yashika arora, Ashi Gupta, and Manjot Kaur Bhatia. cloud computing. *International Journal For Science Technology And Engineering-Vol. 10, Iss : 12, pp 758-761* dec 2022. <https://doi.org/10.22214/ijraset.2022.48010>.
- [25] Ratnakumari Challa and Kanusu Srinivasa Rao. Services of cloud computing. *International Research Journal of Modernization in Engineering Technology and Science. Vol : 04, Iss : 11, pp 1340-1345.* nov 2022. <https://doi.org/10.56726/IRJMETS31489>.
- [26] N. Sultan. Making use of cloud computing for healthcare provision : Opportunities and challenges, *international journal of information management*, Vol 34, Iss 2,pp 177–184. apr 2014. <https://doi.org/10.1016/j.ijinfomgt.2013.12.011>.
- [27] Simon Dupont. *Gestion autonome de l'élasticité multi-couche des applications dans le Cloud : vers une utilisation efficiente des ressources et des services du Cloud.* PhD thesis, Ecole des Mines de Nantes, 2016.
- [28] P. Mell and T. Grance. The nist definition of cloud computing. *Recommendations of the National Institute of Standards and Technology*, 2011.
- [29] E. Venkata. Cloud computing, mobile cloud computing and its comparative study. *Venkata Prabhakar Estamsetty Computing Department Bournemouth University Bournemouth, England*, jan 2021. <https://doi.org/10.13140/RG.2.2.30812.41601>.
- [30] Anirban Sarkar Sugyan Kumar Mishra. Service-oriented architecture for internet of things : A semantic approach. *ournal of King Saud University –Computer and Information Sciences, Volume 34, Issue 10, Part A, Pages 8765-8776*, 2021. <https://doi.org/10.1016/j.jksuci.2021.09.024>.
- [31] Fernando Luis-Ferreira Diogo Gonçalves Joao Sarraipa Joao Giao, Artem A. Nazarenko . A framework for service-oriented architecture (soa)-based iot application development. 2022. <https://doi.org/10.3390/pr10091782>.
- [32] Ismael Ali, R. A. Dela Fuente, J. M. Almufti, S., Ahmad Shaban, A. (2023). overview of metaheuristic algorithms. *Polaris Global Journal of Scholarly Research and Trends*, 2(2), 10–32. <https://doi.org/10.58429/pgjsrt.v2n2a14>.
- [33] Godinez, A. C., Espinosa, L. E. M, and Montes, E. M. An experimental comparison of multiobjective algorithms : Nsga-ii and omopso. 2010.
- [34] Sierra, M. R. and Coello Coello, C. A. Improving pso-based multi-objective optimization using crowding, mutation and -dominance. *Evolutionary Multi-Criterion Optimization*, 505–519. 2005. https://doi.org/10.1007/978-3-540-31880-4_35.

- [35] Manel Boutarfa, Ramdane Maamri, and Mohammed Nassim Lacheheb. Towards an approach for cloud service composition in multi-cloud environment based qos using deep q-learning. *International Conference on Advanced Aspects of Software Engineering (ICAASE)*. octobre 2022. <https://doi.org/10.1109/ICAASE56196.2022.9931591>.
- [36] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2016. <https://doi.org/10.48550/arXiv.1509.06461>.
- [37] Eugenio Bargiacchi Johan Källström Matthew Macfarlane Al. Conor F. Hayes, Roxana Rădulescu. "a practical guide to multi-objective reinforcement learning and planning". *Auton Agent Multi-Agent Syst* 36, 26 (2022). <https://doi.org/10.1007/s10458-022-09552-y>.
- [38] Junge Zhang Qiyue Yin Kaiqi Huang Guangkai Yang, Hao Chen. "multi-agent uncertainty sharing for cooperative multi-agent reinforcement learning", *International Joint Conference on Neural Networks (IJCNN)*. 2022. <https://doi.org/10.1109/IJCNN55064.2022.9891948>.
- [39] Yun Zhang and Bin Jiao. "improved nsga-ii algorithm for optimization of constrained functions". *International Conference on Electrical, Automation and Mechanical Engineering* (2018). <https://doi.org/10.2991/eeame-18.2018.67>.
- [40] Ahuja, K. Sipper, M., Fu, W. "investigating the parameter space of evolutionary algorithms". *biodata mining* 11, 2 (2018). <https://doi.org/10.1186/s13040-018-0164-x>.
- [41] Julian Blank, Kalyanmoy Deb, and Proteek Chandan Roy. "investigating the normalization procedure of nsga-iii". in *evolutionary multi-criterion optimization*. 2019. https://doi.org/10.1007/978-3-030-12598-1_19.
- [42] Rajania, Dinesh Kumarb, and Vijay Kumarc. Impact of controlling parameters on the performance of mopso. vol : 167, pp 2132-2139, 2020. <https://doi.org/10.1016/j.procs.2020.03.261>.
- [43] Dayan, P Watkins, C.J.C.H. Q-learning. *mach learn* 8, 279–292 (1992). <https://doi.org/10.1007/BF00992698>.
- [44] Ann Nowé Mathieu Reymond, Eugenio Bargiacchi. "pareto conditioned networks", *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2022. <https://doi.org/10.48550/arXiv.2204.05036>.
- [45] Matlab - wikipedia. <https://en.wikipedia.org/wiki/MATLAB>. (Consulté le : 23/05/2024).

- [46] Mo-gymnasium documentation. <https://mo-gymnasium.farama.org/index.html>. (Consulté le 30/04/2024).
- [47] Lucasalegre/morl-baselines : Multi-objective reinforcement learning algorithms implementations. <https://github.com/LucasAlegre/morl-baselines>. (Consulté le 30/04/2024).
- [48] Weights & biases : The ai developer platform. <https://wandb.ai/site>. (Consulté le : 17/05/2024).
- [49] pymoo : Multi-objective optimization in python. <https://pymoo.org/#:~:text=Our%20framework%20offers%20state%20of,as%20visualization%20and%20decision%20making>. (Consulté le : 17/05/2024).
- [50] Html - wikipedia. <https://en.wikipedia.org/wiki/HTML>. (Consulté le 30/04/2024).
- [51] Css - wikipedia. <https://en.wikipedia.org/wiki/CSS>. (Consulté le 30/04/2024).
- [52] Javascript - wikipedia. <https://en.wikipedia.org/wiki/JavaScript>. (Consulté le 30/04/2024).
- [53] The web framework for perfectionists with deadlines | django. <https://www.djangoproject.com/>.
- [54] Chaojing Yao. Learning an interpretable learning rate schedule via the option framework *IEEE 34th International Conference on Tools with Artificial Intelligence (ICTAI)* .2022. <https://doi.org/10.1109/ICTAI56018.2022.00111>.
- [55] Matthieu Basseur, Rong-Qiang Zeng, and Jin-Kao Hao. Hypervolume-based multi-objective local search. <https://doi.org/10.1007/s00521-011-0588-4>, 2011. Consulté le : 06/05/2024.

Annexe A

Implementation de l'algorithme NSGA-III

Comme mentionné dans le troisième chapitre, NSGA-III est un algorithme métaheuristique évolutif multi-objectifs qui se démarque par son efficacité remarquable lorsqu'il est appliqué à des problèmes comportant un grand nombre d'objectifs.

Nous avons tenté d'implémenter NSGA-III principalement pour le comparer aux résultats obtenus par l'apprentissage par renforcement. Nous avons constaté des améliorations significatives en termes de temps d'exécution : il a fallu seulement une minute et 31 secondes pour atteindre la fin des itérations.

A.0.1 Paramètres

Après avoir mené plusieurs tests, nous avons sélectionné ces paramètres en raison de leur performance favorable et de la durée courte qu'ils ont permis d'atteindre.

Paramètre	Valeur attribué
Taille population	700
Nombre maximal de générations	15
Taux de croisement	(1.0,30)
Taux de mutation	(0.7,20)

Tableau A.1: Paramètres attribués au NSGA-III

A.0.2 Métriques

Ci-dessus, nous présentons les métriques obtenues pour deux compositions de services $[S_0S_1S_2]$ et $[S_0S_4S_5]$. Nous les avons regroupées dans un même graphique afin de fournir une comparaison visuelle.

Métriques		
Composition	Hypervolume	IGD
$[S_0 S_1 S_2]$		
$[S_0 S_4 S_5]$		

A.0.3 Graphe de nuage de points par paire

En utilisant MATLAB, nous avons tracé les fonctions objectifs deux par deux. Cette approche visuelle nous a permis de comprendre les résultats obtenus par l'algorithme de manière claire et intuitive. Dans les tracés suivants, Les points sont représentés comme suit :

- **En rouge** : Les points du vrai Pareto identifiés par la règle de dominance.
- **En vert** : Les points trouvés par MPMOQL.
- **En bleu** : Les points du vrai Pareto trouvés par MPMOQL.

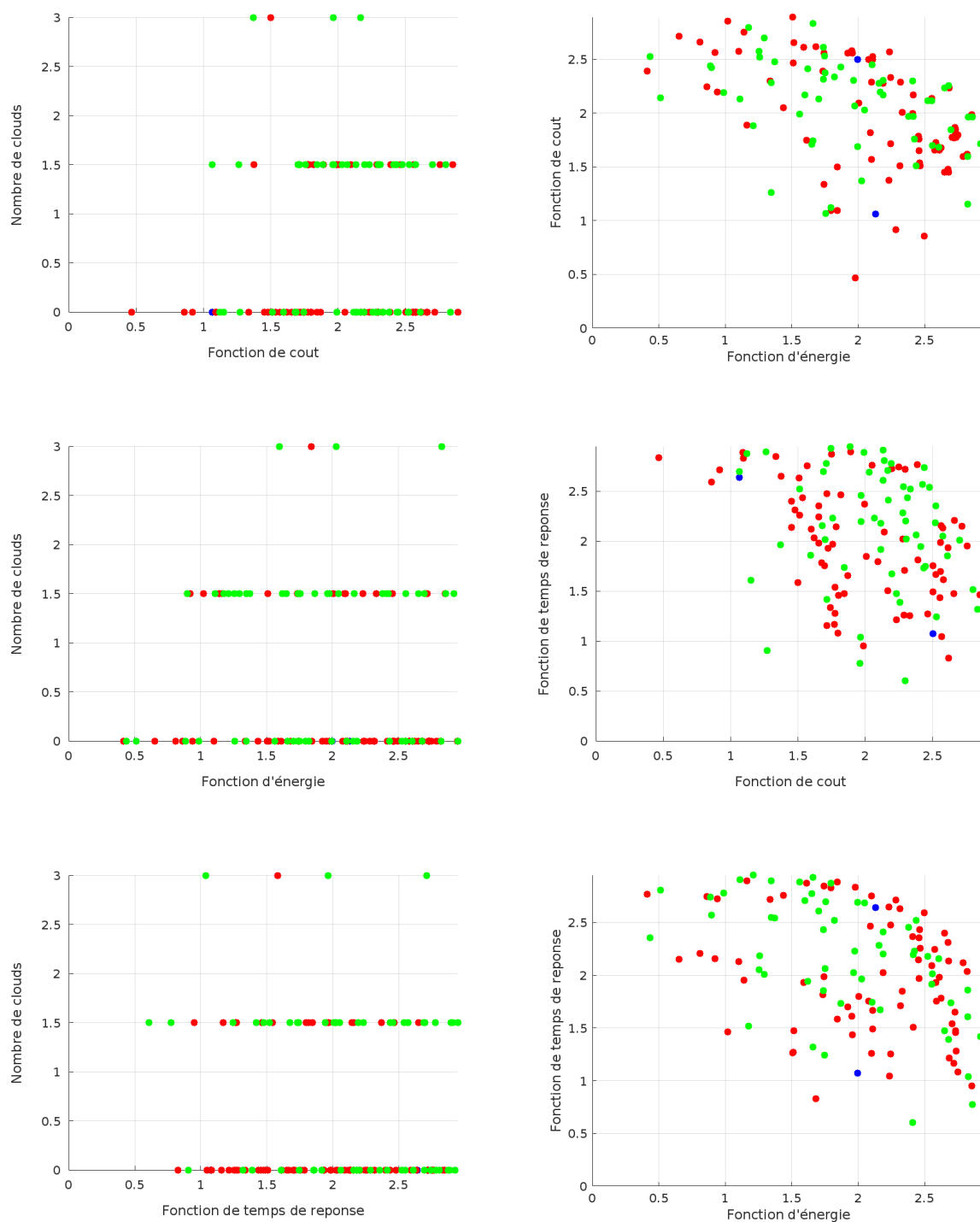


Figure A.1: Graphe de nuage de points par paire

A.0.4 Comparaison

À la fin, NSGA-III nous a donné de meilleurs résultats, surtout en comparant l'IGD des deux approches. NSGA-III nous donne clairement des résultats de plus en plus proches du vrai Pareto. Ceci peut être expliqué par plusieurs raisons :

- NSGA-III est beaucoup plus documenté, avec plus d'articles disponibles qui expliquent bien chaque paramètre.
- La bibliothèque PYMOO est bien conçue, facile à utiliser et permet une implémentation simple des métaheuristiques.
- NSGA-III a déjà été testé avec un grand nombre d'objectifs, ce qui en fait un algorithme considéré comme "many-objectives".

Cette comparaison met en évidence les avantages de NSGA-III par rapport à l'approche précédente, notamment en termes de précision et de facilité d'utilisation.

Annexe B

Tests supplémentaires

Nous avons choisi de réaliser des tests supplémentaires sur MPMOQL, car il donne de meilleurs résultats que PCN. C'est pourquoi nous le considérons comme notre algorithme principal.

Les tests supplémentaires sont essentiels pour plusieurs raisons. Ils permettent de confirmer la robustesse et la fiabilité de l'algorithme dans des scénarios variés, garantissant ainsi des performances optimales dans différentes conditions. De plus, ils aident à identifier et corriger les éventuelles faiblesses ou anomalies qui pourraient ne pas être apparues lors des tests initiaux.

B.0.1 Métriques

Ci-dessus, nous présentons les métriques obtenues pour 10 compositions de services supplémentaires. Nous les avons regroupées dans un même graphique afin de fournir une comparaison visuelle.

B.0.1.1 Hypervolume

Nous remarquons que l'hypervolume varie légèrement en fonction des services choisis dans les compositions, se situant approximativement entre 120 et 180, comme indiqué sur le graphique ci-dessus.

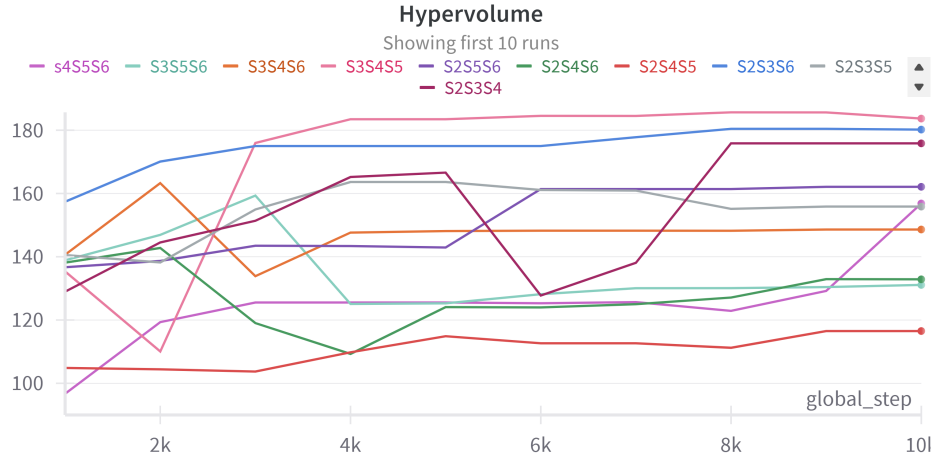


Figure B.1: Graphique représentant l'hypervolume de dix compositions différentes.

B.0.1.2 Inverted generational distance (IGD)

À travers le graphique ci-dessous, nous constatons que l'IGD se situe généralement entre 0,4 et 0,6. Les solutions avec un IGD plus faible indiquent une distance moindre par rapport au vrai front de Pareto, ce qui correspond à une meilleure optimisation.

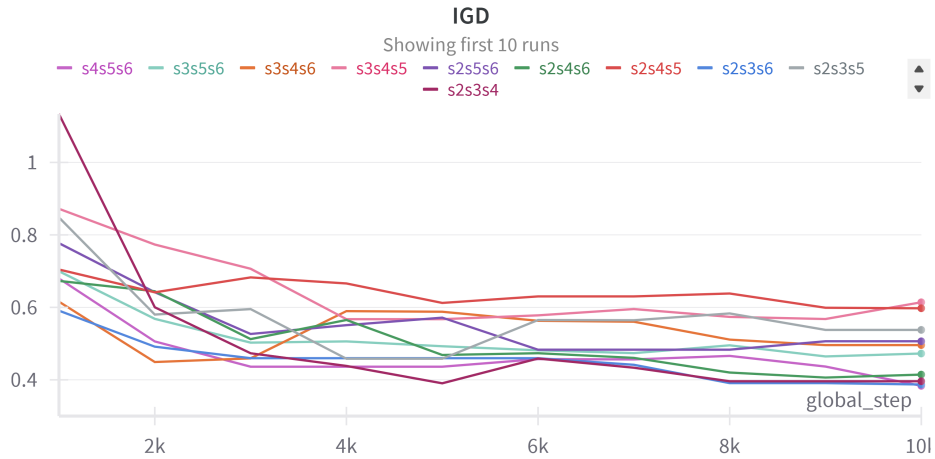


Figure B.2: Graphique représentant l'IGD de dix compositions différentes

B.0.1.3 Cardinalité

Nous remarquons que la cardinalité est entre 15 et 40, les solutions avec cardinalité plus élevée indiquent l'existence des solutions diversifiées dans l'espace objectif.

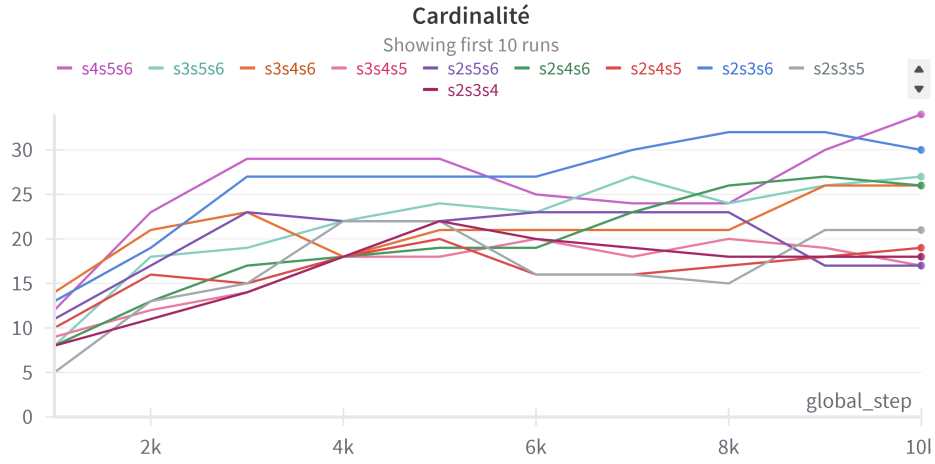


Figure B.3: Graphique représentant la cardinalité de dix compositions différentes

B.0.2 Graphe de nuage de points par paire

En utilisant MATLAB, nous avons tracé les fonctions objectifs deux par deux. Cette approche visuelle nous a permis de comprendre les résultats obtenus par l'algorithme de manière claire et intuitive. Dans les tracés suivants, Les points sont représentés comme suit :

- **En rouge** : Les points du vrai Pareto identifiés par la règle de dominance.
- **En vert** : Les points trouvés par MPMOQL.
- **En bleu** : Les points du vrai Pareto trouvés par MPMOQL.

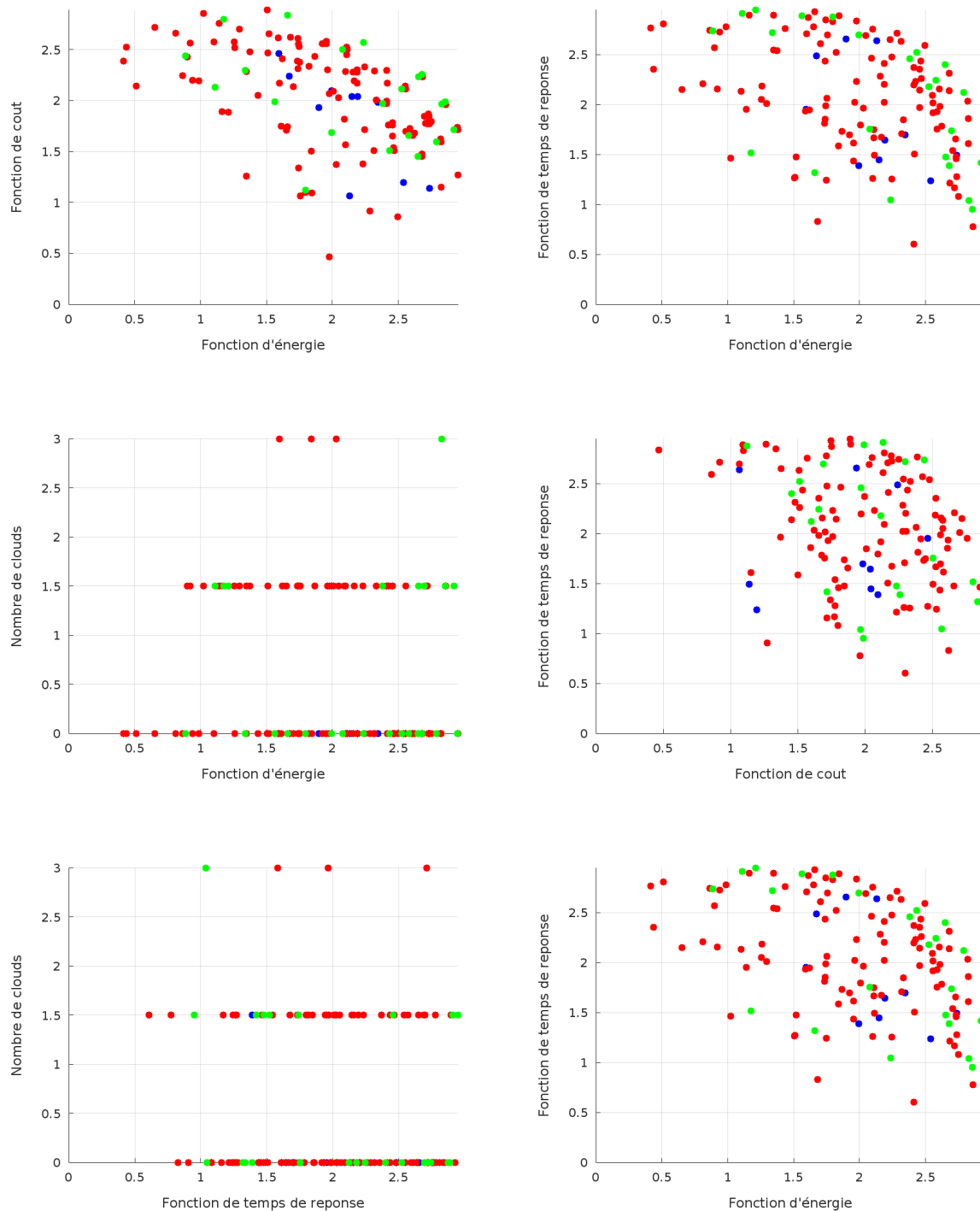


Figure B.4: Graphe de nuage de points par paire

Annexe C

Algorithme Envelope

L'algorithme Envelope, une méthode d'optimisation multi-objectifs par renforcement profond. L'un des premiers algorithmes que nous avons tenté d'implémenter. Nous l'avons choisi pour sa relative simplicité et sa forte ressemblance avec DQN.

DQN étant bien documenté et disposant de nombreuses informations disponibles, la transition de DQN à Envelope est facilitée. En effet, ces deux algorithmes partagent de nombreux hyperparamètres, ce qui simplifie grandement le processus d'implémentation.

C.1 Hyperparamètres

Nous avons tester Envelope avec des composition de trois services et cinque clouds seulement, Après avoir mené plusieurs tests, nous avons sélectionné ces paramètres :

Hyperparamètre	Valeur attribuée
Taux d'apprentissage	0.003
Étapes de décroissance d'epsilon	50,000
Facteur de remise	1
Pas de temps par itération	1000
Pas de temps total	100,000
Taille du lot	64
Taille maximale du tampon	2,000,000

Tableau C.1: Hyperparamètres attribués pour l'algorithme Envelope

C.2 Tests et analyse

Nous avons obtenu les métriques suivantes lors de l'entraînement de l'algorithme Envelope pour la composition de services $[S_0S_1S_2]$.

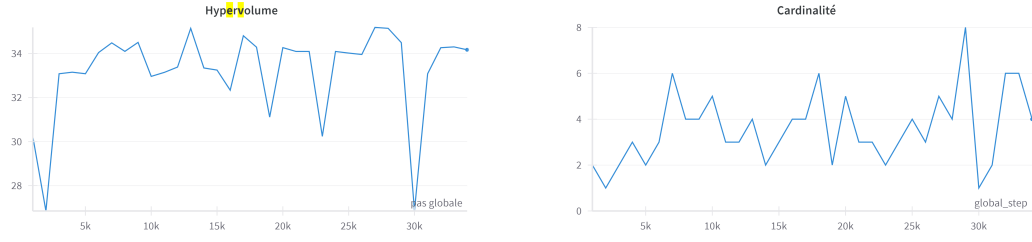


Figure C.1: Hypervolume et Cardinalité de l'algorithme Envelope

C.3 Discussion

L'expérience avec l'algorithme Enveloppe n'a pas produit des résultats satisfaisants, l'agent n'a pratiquement pas progressé. Cette situation pourrait résulter de choix inappropriés d'hyperparamètres, étant donné que cet algorithme nécessite un temps considérable pour s'exécuter. Par conséquent, l'ajustement des hyperparamètres était une tâche complexe.